

BACHELOR'S THESIS

International IT Management: Major Technology

Developing an Algorithmic Solution to the Vehicle Routing Problem with Time Windows for a Bakery Using Google OR-Tools

DECEMBER 10, 2025

STUDENT: KRISH SHARMA

FIRST ADVISOR: PROF. DR. HABIL. Jana Koehler

SECOND ADVISOR: Prof. René Meier

Table of Contents

1. Abstract	5
2. Introduction and Problem Statement	6
3. Literature Review	8
3.1 Foundations of Routing Optimization	8
3.1.1 The Traveling Salesperson Problem (TSP)	8
3.1.2 Generalization to Logistics: VRP	8
3.1.3 The Vehicle Routing Problem with Time Windows (VRPTW)	9
3.2 Computational Implementation: Google OR-Tools	9
3.3 Constructive Heuristics and Metaheuristics	10
3.3.1 Constructive Heuristics (First Solution Strategy)	10
3.3.2 Metaheuristics (Local Search)	11
3.4 Alternative Modelling and Solving Approaches	12
4. Methodology and Implementation	12
4.1 Data Acquisition and Initial Description	12
4.2 Data Quality Assessment	14
4.2.1 Structural and Formatting Issues	14
4.2.2 Incompleteness and Inconsistency	14
4.3 Data Cleaning and Preprocessing Strategies	16
4.3.1 Restructuring and Standardization	16
4.3.2 Enrichment and Validation	16
4.3.3 Iterative Geocoding Correction	16
4.4 Geocoding and Place ID Extraction	17
4.4.1 Request Logic and Data Extraction	18
4.4.2 Ambiguity Handling	19
4.4.3 Error Handling and Status Validations	19
4.4.4 Data Storage and Export	19
4.5 Duration Matrix Calculation	20
4.5.1 API Choice and Traffic Prediction	20
4.5.2 Batching and Matrix Construction	21
4.5.3 Fault Tolerance and Checkpointing	21
4.5.4 Final Output	21
4.6 Solving the VRPTW	22
4.6.1 Model Initialization	22
4.6.2 Time Dimension and Constraints	23

4.6.3 Dropping Customers (Penalty Mechanism)	24
4.6.4 Search Strategy Configuration.....	24
4.6.5 Output Interpretation	25
4.7 Process Visualization and Workflow Summary	25
4.7.1 High-Level Solution Architecture	25
4.7.2 Data Preprocessing Subprocess	26
5. Results.....	27
5.1 Performance Analysis.....	28
5.1.1 Convergence at 25,629	28
5.1.2 The Performance of Guided Local Search (GLS).....	28
5.1.3 Best Configuration.....	28
5.2 Final Routes and Schedules.....	29
5.3 Comparative Analysis: Algorithmic vs. Manual Planning	30
5.3.1 Baseline Manual Schedule	30
5.3.2 Performance Metrics and Efficiency Gains.....	31
5.4 Operational and Algorithmic Outcomes	31
5.6.1 Analysis of Idle Time	31
5.6.2 Route Imbalance	32
5.7 Hardware Comparison and Computational Environment.....	32
6.0 Budget Considerations	33
6.1 Operational Demand Analysis.....	33
6.2 Component Cost Structure	34
6.2.1 Variable API Costs (Google Maps Platform)	34
6.2.2 Fixed Infrastructure and Operational Labor.....	35
6.3 Operational Scenarios	35
6.3.1 Scenario A: Nightly Static Solution.....	35
6.3.2 Scenario B: Ad-Hoc On-Demand Solution.....	36
6.3.3 Scenario C: Real-Time Dynamic Solution	36
6.4 Operational Solution Recommendation.....	37
7. Evaluation and Future Work.....	37
7.1 Critical Evaluation of Research Questions.....	37
7.1.1 RQ1: Solution Quality and Computational Efficiency.....	37
7.1.2 RQ2: Scalability to Demand.....	38
7.1.3 RQ3: Budgetary Implications.....	38
7.2 Operational Limitations.....	38

7.3 Future Work	39
7.3.1 Algorithmic Refinements.....	39
7.3.2 System Integration and Automation	39
7.3.3 Cost Optimization via Caching.....	40
7.3.4 Benchmarking with Exact Solvers	40
8. Conclusion.....	40
9. References.....	42
10. Appendix	44

Table of Figures

Figure 1: Manual delivery schedule for the "Lake Tour" (ZH1 See)	13
Figure 2: Manual delivery schedule for the "City Tour" (ZH2 Stadt)	13
Figure 3: Raw data excerpt highlighting quality issues,	14
Figure 4: Excerpt illustrating data inconsistencies,	15
Figure 5: Excerpt showing critical logistical dependencies	15
Figure 6: Excerpt highlighting unstructured constraints,	15
Figure 7: Processed dataset following restructuring and enrichment.	17
Figure 8: Python implementation of the geocode_address function (Part 1).	18
Figure 9: Python implementation of the geocode_address function (Part 2).	19
Figure 10: Python logic for calculating a predictive departure time.....	20
Figure 11: The core matrix construction loop.....	22
Figure 12: Code snippet showing the initialization of the RoutingIndexManager and RoutingModel,	23
Figure 13: Configuration of the Time Dimension using AddDimension.	23
Figure 14: Solver configuration snippet.	25
Figure 15: High-level BPMN diagram	26
Figure 16: Detailed BPMN subprocess for Data Preprocessing.	27
Figure 17: Distribution of confirmed deliveries per day of the week.	34

List of Tables

Table 1: A snippet of the worst and best performing configurations	28
Table 2: Monday delivery routes and schedule for both delivery vans.	29
Table 3: Current Manual Delivery Plan (Baseline).....	30
Table 4: Hardware Specifications and Performance Benchmarks.....	33

Declaration of Independent Work:

"I affirm that this Bachelor thesis was written by myself without any un-authorized third party support. All used references and resources are clearly indicated. All quotes and citations are properly referenced. This thesis was never presented in the past in the same or similar form to any examination board. I agree that my thesis may be subject to an electronic plagiarism check. For this purpose, an anonymous copy may be distributed and uploaded to servers within and outside the LUASA."

Rotkreuz, 10th of December, 2025

(Signature):  _____

1. Abstract

MSMEs often rely on inefficient manual planning for their logistics due to the high cost and complexity of commercial routing software. This study addresses the central question of whether accessible, open-source optimization tools can provide a scalable alternative that effectively manages complex constraints, specifically strict delivery time windows. The research focuses on a case study of a Swiss bakery to validate the hypothesis that an algorithmic solution using Google OR-Tools can significantly outperform traditional manual routing methods.

The methodology involved the development of a custom Python-based algorithmic pipeline. This process began with a rigorous data cleaning and geocoding phase using the Google Maps API to resolve inconsistent addresses into precise Place IDs. A Duration Matrix was generated using predictive traffic modeling to ensure realistic travel time estimates. The core routing problem was formulated as a Vehicle Routing Problem with Time Windows (VRPTW) and solved using the Google OR-Tools library. An experimental framework tested various combinations of constructive heuristics (e.g., Global Cheapest Arc) and metaheuristics (e.g., Guided Local Search) to identify the most effective configuration.

The results demonstrate a substantial efficiency gap between human intuition and algorithmic precision. The algorithmic schedule reduced the vehicle's total active-duty time by 30.74%, saving approximately 3.5 labor hours per day compared to the manual baseline. While the total distance traveled increased slightly by 6.37% or approximately 5 km, this reflected the solver's strategic preference for faster road networks to meet tight deadlines. Furthermore, a cost analysis confirmed the system's financial viability, estimating a monthly operating cost of approximately 485 CHF—a fraction of the value saved in labor and vehicle wear. This study confirms that heuristic solvers offer a robust, cost-effective pathway for MSMEs to transition from manual to automated dynamic logistics.

2. Introduction and Problem Statement

Millions of Micro, Small, and Medium-Sized Enterprises (MSMEs) deliver physical goods to customers every day. Whether they work with fast-moving consumer goods, industrial equipment, or personalized services, they all face a common logistical challenge: getting the right products to the right customers at the right time.

Achieving this goal is not always difficult. The difficulty depends on scale and other factors. For a business with a single vehicle and a handful of local stops, planning a route is intuitive. But the challenge becomes harder as operations grow. When a company must work with multiple vehicles, dozens of delivery points, and tight time windows, the logistics also becomes more complex. It is at this level of complexity that manual planning begins to fail, and the need for mathematical algorithms becomes critical.

These forms of optimization problems are known as Vehicle Routing Problems (VRPs) (Saller et al., 2023). The difficulty of the problem is further enhanced as MSMEs must account for many changing factors while searching through an enormous number of possible delivery routes. The goal is to find the best routes for a fleet of vehicles while minimizing distance, time, or cost. Since the number of possible routes grows extremely quickly as more stops are added, most real-world systems require specialized commercial optimization tools. These specialized tools for large scale operations can be quite expensive to run and integrate into the current working system. Therefore, many MSMEs still rely on manually planned delivery routes.

To comprehend the theoretical complexity underlying these logistical challenges, one must first examine the foundational model upon which VRPs are built.

Understanding the Traveling Salesman Problem (TSP) is the baseline for understanding VRPs. Quoted directly from Saller et al., 2023, the TSP is:

“Given a list of cities and distances between them, a traveling salesperson needs to find a shortest possible route to visit the cities” (pg. 7)

The VRP expands this idea by introducing multiple vehicles and additional restrictions for its variants. In contrast to the TSP’s single vehicle with unlimited capacity, real operations involve limited fleets, restricted capacities, and other practical constraints (Saller et al., 2023). Thus, the VRP is generally defined as:

“... design of a set of minimum-cost vehicle routes, originating and terminating at a central depot, for a fleet of vehicles that services a set of customers with known demands. Each customer is serviced exactly once and, furthermore, all the customers must be assigned to vehicles without exceeding vehicle capacities.” (Salomon, 1987, pg. 254)



This paper focuses on solving a variant of the VRP for a bakery factory in Switzerland. To do this, it is, first, necessary to understand the factory’s operations and requirements.

The main factory or depot, located in Lanterswil, supplies to customers in the Zurich region, including bakeries, cafés, markets, and other food retailers. The depot handles two types of deliveries: freshly baked goods and frozen goods. Deliveries of freshly baked goods must ideally be completed before stores open, while deliveries of frozen goods must occur during normal business hours. However, some customers require deliveries within a specific time window. The depot currently operates two vans, which are not used to full capacity, and makes deliveries six days a week. The current schedule for the vehicles is produced manually by an employee on an excel sheet.

The biggest challenge faced by the depot currently is the daily changing customer list. For each delivery day in a given week, the customer list changes, which requires a renewal of the route and schedule of the vehicles. Trying to handle these changes, while abiding by the time windows of the customers and factoring in any spontaneous traffic delays, manually can be extremely time-consuming and complex.

To handle these logistical challenges without relying on expensive enterprise software, this study uses Google OR-Tools, an open-source optimization library designed for solving complex routing problems. Unlike many commercial systems that are often rigid and costly, OR-Tools offers a flexible Routing Library that can model detailed real-world constraints—such as the bakery’s changing daily demand and strict delivery time windows—while remaining affordable. Because it is both powerful and customizable, OR-Tools represents a practical and accessible solution for MSMEs that want to automate their routing processes and adapt the system to their specific local needs.

Given these requirements, the Vehicle Routing Problem with Time Windows (VRPTW) becomes the central theoretical framework for this work (Desrochers et al., 1988).

Thus, the aim of this paper is to answer the following questions:

- RQ1: Can the search algorithms provided by Google OR-Tools find correct solutions of good quality in reasonable time?
- RQ2: Is the solution scalable to current and future demand?
- RQ3: What are the budgetary implications of maintaining this system?

3. Literature Review

3.1 Foundations of Routing Optimization

To address the logistical challenge faced by the bakery in Lanterswil and to move from manual scheduling to an algorithmic solution, the approach must be grounded in established combinatorial optimization theory. Although the bakery has a specific requirement, time windows, the core of the challenge is rooted in two key mathematical problems: TSP and VRP

3.1.1 The Traveling Salesperson Problem (TSP)

As discussed earlier, the TSP is the foundational model for understanding transport and routing logic. Formally, it is represented as a graph, $G = (V, E)$ where, V , is the set of locations (cities or customers) and E is the set of roads connecting them.

The objective is to find a Hamiltonian cycle—a circular route that visits every location exactly once—while minimizing total travel distance or cost. (Saller et al., 2023)

Although the TSP appears simple, it is classified as NP-hard, meaning no known algorithm can solve it efficiently for large input sizes (Wolfram Research, n.d.). Efficient algorithms usually run in polynomial time, meaning the time required increases at a manageable rate as the problem size grows. In contrast, NP-hard problems like the TSP become far more demanding because the number of possible routes grows factorially, following a complexity on the order of $O(n!)$. As a result, even a small increase in the number of customers causes the computational effort to explode. For example, a route with only 20 stops already produces more than 6×10^{16} (60 quadrillion) possible route combinations (Jia et al., 2023). This rapid combinatorial growth makes exact optimization methods unrealistic for expanding MSMEs unless optimization algorithms are used.

3.1.2 Generalization to Logistics: VRP

To make routing models more useful for real logistics operations, the TSP was extended into the Vehicle Routing Problem (VRP). The VRP includes a fleet of k vehicles, usually with limited capacity, instead of just one. The goal becomes finding a set of routes that services all customers while minimizing total distance, time, or cost. This shift makes the model much more representative of real business operations, such as multi-truck deliveries from a single depot (Dantzig and Ramser, 1959).

3.1.3 The Vehicle Routing Problem with Time Windows (VRPTW)

In the case of the Lanterswil depot, simple distance-based routing is not enough. The bakery must meet strict customer time windows, fresh goods must be delivered before shops open, and frozen goods must arrive during business hours. These constraints transform the problem into a Vehicle Routing Problem with Time Windows (VRPTW).

The VRPTW adds timing constraints to the routing structure. Each customer i has an assigned time interval $[a_i, b_i]$. A vehicle cannot start service before a_i , and usually cannot arrive after b_i . Desrochers et al. (1988) define the VRPTW as follows:

“A number of vehicles is located at a single depot and must serve a number of geographically dispersed customers. Each vehicle has a given capacity. Each customer has a given demand and must be served within a specified time window. The objective is to minimize the total cost of travel.”

This aligns closely with the bakery’s real operations, unlike typical VRPTW cases where vehicle capacity is the main limiting factor, the bakery’s biggest constraint is the strict customer time windows rather than the amount of goods the vehicles can carry. Time windows make the routing problem significantly harder. The shortest geographical route might not be feasible if the delivery time does not align with the customer’s requirements. Arriving too early may force the vehicle to wait and arriving too late results in a failed delivery. Because timing and routing interact so tightly, even a small change in the schedule can disrupt the entire route structure (Bräysy & Gendreau, 2005a).

3.2 Computational Implementation: Google OR-Tools

To solve the VRPTW while staying within the budget limits of an MSME, open-source tools are a practical alternative to expensive commercial logistics software. Google OR-Tools (Furnon & Perron, 2025) is one of the most widely used open-source optimization frameworks. Its Routing Library `ortools.constraint_solver.routing` abstracts away most of the complex constraint-programming logic, making it easier to model and solve routing problems like the one faced by the bakery.

The OR-Tools Routing Library runs on top of a Constraint Programming (CP) solver (accessed via `pywrapcp`). One of its key features is the use of Dimensions, which track cumulative quantities along a vehicle’s route (Google, 2024).

In many VRP applications, the Capacity Dimension is essential for ensuring that vehicles do not exceed their loading limits. However, the Lanterswil bakery does not reach full van capacity, so this dimension plays little role here. Instead, this study focuses on the Time Dimension, which tracks travel time, service duration, and enforces customer time windows $[a_i, b_i]$.

Although the Time Dimension keeps routes within valid time windows, waiting time is not normally penalized in the objective function. As a result, the solver may create routes where drivers arrive early and wait at customer locations. This is a trade-off: the solver accepts idle time if it helps minimize travel time while respecting all time-window constraints.

3.3 Constructive Heuristics and Metaheuristics

Solving a VRPTW exactly is computationally demanding. OR-Tools addresses this by using a two-step search: first it creates a feasible starting solution using constructive heuristics, and then it improves this solution using metaheuristics (Google, 2025).

3.3.1 Constructive Heuristics (First Solution Strategy)

These heuristics quickly generate an initial, valid route (Bräysy & Gendreau, 2005a):

- **Path Cheapest Arc:** This method is like the classic Nearest-Neighbour strategy, but instead of simply choosing the geographically closest stop, OR-Tools selects the next customer based on the lowest cost according to its internal cost function. This cost can include travel time, distance, or penalties, making it more flexible than the standard version.
- **Global Cheapest Arc:** This approach looks at the entire graph and repeatedly connects the two nodes that form the cheapest valid link, no matter which vehicle is involved. It continues adding the globally cheapest arcs as long as the resulting connections do not violate any constraints.
- **Savings Heuristic:** Based on the Clarke and Wright (1964) method, this algorithm begins by assigning each customer to their own individual route. It then merges routes whenever doing so produces a “saving” — meaning the combined route reduces the overall cost compared to keeping them separate. Over time, these merges build efficient multi-stop routes.
- **Parallel Cheapest Insertion:** This strategy constructs several routes at the same time. At each step, it searches for the unvisited customer whose insertion into any existing route increases the cost the least. This parallel structure helps distribute stops more evenly across vehicles, making it especially suitable for problems with strict time windows.

- Christofides: This version is adapted from the Christofides (1976) algorithm for the TSP. Although the classic method provides a 1.5-approximation guarantee for metric problems, OR-Tools uses a faster maximal matching procedure instead of the optimal maximum matching. The implementation is also sequential: it builds one route using Christofides logic until no more customers can be added without violating constraints, and then it moves on to form the next route for the remaining customers (Google, 2024-b).

3.3.2 Metaheuristics (Local Search)

To enhance the schedule further, the solver employs an improvement phase. The user must select one strategy to govern this process. Although the Google OR-Tools library groups all these options under the single configuration parameter LocalSearchMetaheuristic (Google, 2024-b), academically they represent two distinct approaches: deterministic local search (Greedy Descent) versus metaheuristics (GLS, SA, Tabu).

A. Deterministic Local Search

- Greedy Descent: The default iterative method. It explores the neighbourhood of the current solution and accepts the first move that improves (reduces) the objective cost. It is computationally fast but stops immediately upon reaching a local optimum—a solution that is better than its neighbours but potentially worse than the global best (Bräysy & Gendreau, 2005-a).

B. Metaheuristics To overcome the limitations of getting trapped in local optima, the following metaheuristics can be selected to guide the search process (Bräysy & Gendreau, 2005-b):

- Guided Local Search (GLS): A metaheuristic that prevents the solver from getting trapped by augmenting the objective function with penalty terms for specific features (like long travel legs). This dynamically changes the search landscape to force the solver out of local traps. Google OR-Tools notes this is generally the most efficient metaheuristic for vehicle routing.
- Simulated Annealing (SA): Inspired by the metallurgical process of annealing, SA allows the algorithm to probabilistically accept worse solutions during the search. This randomness prevents the solver from getting stuck in a sub-optimal solution early in the process.
- Tabu Search: This method utilizes memory structures (a "tabu list") to forbid the reversal of recent moves for a certain number of iterations, forcing the exploration of new, previously unvisited routes.

3.4 Alternative Modelling and Solving Approaches

Although this study uses the heuristic features of Google OR-Tools, several other well-established modelling approaches exist for solving the VRPTW.

One of the most traditional methods is to formulate the problem as a Mixed Integer Linear Program (MILP), where binary decision variables determine the routes. For example, Pacheco et al. (2012) successfully applied a MILP model to optimize delivery routes for a bakery in Northern Spain. MILP solvers such as Gurobi and CPLEX can generate exact, globally optimal solutions, but they often struggle to scale. As the number of customers grows, the computational effort required to guarantee optimality increases sharply, making MILP difficult to use in a fast-moving daily operation.

Another approach is Constraint Programming (CP), which emphasizes satisfying complex logical constraints rather than optimizing a linear objective. CP solvers like CP-SAT or Choco are particularly strong when dealing with intricate real-world rules—for example, mandatory driver breaks, special delivery restrictions, or multi-day schedules (Rossi et al., 2006).

Although both exact MILP and CP models could technically be implemented at the bakery's current scale, they require more development effort and ongoing maintenance compared to the OR-Tools Routing Library. OR-Tools abstracts many of the constraints and provides a flexible foundation that allows for fast development and easier updates. For this reason, the study uses the heuristic approach to deliver a practical, reliable solution that balances strong performance with day-to-day usability. However, once the bakery's operations become more stable, building a custom exact solver to benchmark against the heuristic results would be a logical direction for future research and system improvement.

4. Methodology and Implementation

Building on the theoretical foundations established in the literature review, this chapter explains the practical steps taken to implement the algorithmic solution. Since every routing algorithm depends on high-quality structured input, the first phase involved collecting, auditing, and preparing the bakery's data so that it could be geocoded and used in the Google OR-Tools solver.

4.1 Data Acquisition and Initial Description

The bakery provided its current delivery routes and schedules in two separate Excel spreadsheets, representing the two existing static routes: the Lake Tour and the City Tour.

Each entry corresponded to a single customer and included the following information:

- Internal Index: A unique identifier used internally by the bakery.
- Customer Details: A combined field containing both the customer's name and address.
- Service Requirements: A merged column listing store opening times and "TK" flag indicating whether frozen goods were required.
- Departure Time: The planned time at which the driver leaves the customer's location.
- Delivery Schedule: Six columns (Monday–Saturday) indicating which days the customer receives deliveries.
- Remarks: A free-text field used for driver instructions, alternative delivery locations or strict time windows.

13	477	Molli Stäfa, Bahnhofstrasse 22, 8712 Stäfa	6:00	6:05	☐	☐	☐	☐	☐	☐	☐	in den Laden stellen
14	486	Bioladen Stockenweid, Schulhausstrasse 18, 8706 Meilen	6:30	6:15	☐	☐	☐	☐	☐	☐	☐	Haupteingang,
15	526	Kreisladen GmbH, Untere Heslibachstrasse 19, Kusnacht	9:00		☐	☐	☐	☐	☐	☐	☐	in den Laden stellen
16	488	Restaurant Blaue Ente, Seefeldstrasse 223, 8008 Zürich	11:30		☐	☐	☐	☐	☐	☐	☐	Eingang rechts > in die Küche stellen
17	450	Bachsemarkt Seefeld, Seefeldstrasse 29, 8008 Zürich	8:00	6:50	☐	☐	☐	☐	☐	☐	☐	Auf die Theke oder auf Rolli stellen
18	473	Wild Bowls, Seefeldstrasse 25, 8008 Zürich	TK	6:55	☐		XXX					Auf Abruf, ca. 2x pro Monat, Hintereingang
19	255	Tibits Bistro, Falkenstrasse 12, 8008 Zürich	TK	7:00	☐							Anlieferung Muster oder Auslieferer Kappeli hat nichts am Lager
20	408	Reformhaus Stockenstrasse, Stockenstrasse 38, Zürich	8:00	7:10	☐	☐	☐	☐	☐	☐	☐	Hintereingang > beim Ladeneingang auf Rolli stellen
21	451	Bachsemarkt Kalkbreite, Badenerstrasse 171, Zürich	8:00	7:20	☐	☐	☐	☐	☐	☐	☐	Im Laden auf Rolli des Geschäfts stellen
22	484	Restaurant Beke, Bertastrasse 16, 8003 Zürich	10:00	7:25							☐	Ware in der Küche deponieren
23	512	Batter by Sibille Albertin, Hildastrasse 11, Zürich	10:00									Auf Abruf bei Restaurant Beke abgeben
24	276	West 169, Weststrasse 169, Zürich	12:00	7:25							☐	Beim Eingang links
25	267	Wirtschaft im Franz, Bremgartenstrasse 18	TK	7:30	☐							TK in Keller (Kollektiv Weitblick auch dort abladen)
26	268	Der Bioladen, Altstetterstrasse 128, Zürich	9:00	7:45	☐	☐	☐	☐	☐	☐	☐	Mit Rolli in den Laden fahren (369 Bio Hof Zug > auf Abruf dort abladen)

Figure 1: Manual delivery schedule for the "Lake Tour" (ZH1 See) showing customer addresses, opening times, departure times for the vehicle, and logistical remarks including drop-off locations and strict time windows.

13	237	Welschland à la Opfelfascher, Zweierstrasse 56, Zürich	7:00	6:10	☐	☐	☐	☐	☐	☐	☐	Im Laden stellen
14	437	Altstadt Hotel, Kirchgasse 4, 8001 Zürich	6:30	6:20	☐	☐	☐	☐	☐	☐	☐	In den Gang stellen
15	179	Beni Apotheke, Witikonstrasse 299, 8053 Zürich	8:00	6:30							☐	Auf der Rampe im UG > Kiste mit Deckel abdecken
16	474	Bistrot à Paris, Asylstrasse 70, Zürich	11:30	6:40	☐	☐	☐	☐	☐	☐	☐	im Garten beim Eingang links > mit Plastik abdecken
17	355	LPQ, Le Pain Quotidien, Römerhofplatz 5, Zürich	7:30	6:40	☐	☐	☐	☐	☐	☐	☐	ins Restaurant stellen, TK- in Kühler stellen, (SO-TK auf Rolli in TK Boxen lassen)
18	502	Privatbank Bethanien, Toblerstrasse 51, 8044 Zürich		6:45	☐						☐	Beim Lieferanteneingang Küche Türe
19	434	Villa Volta, Voltastrasse 40, Fluntern Zürich	7:00	6:50	☐	☐	☐	☐	☐	☐	☐	Darf nicht vor 06:50 Uhr geliefert werden > im Laden abgeben
20	310	L'Ultimo Bacio, Nordstrasse 227, Wipkingen	7:00	7:00	☐	☐	☐	☐	☐	☐	☐	Im Laden abgeben/Retouren auf Terrasse
21	497	Chez Nous ZH GmbH, Roschibachstrasse 79, Zürich	TK	7:05	☐	☐	☐	☐	☐	☐	☐	Auf Abruf ca. 1x im Monat
22	415	Sphères, Hardturmstrasse 66, Zürich	8:00	7:10	☐	☐	☐	☐	☐	☐	☐	ins Restaurant abgeben. Falls Restaurant geschlossen auf einer leeren Kiste in Gang stellen > bei geschlossenem Restaurant > ALARM
23	427	Tritt-Käse, Limmatstrasse 231, Viadukt, Zürich	8:00	7:15	☐	☐	☐	☐	☐	☐	☐	In Laden stellen
24	428	Südhang Weine, Limmatstrasse 231, Viadukt, Zürich	8:00	7:15		☐						Bei Tritt-Käse separat abgeben
25	469	Onno GmbH, Nordstrasse 89, 8037 Zürich	10:00	7:20	☐	☐	☐	☐	☐	☐	☐	Bei Café des Amis abgeben > separater Stapel
26	524	Café des Amis, Nordstrasse 88, 8037 Zürich	8:00	7:20	☐	☐	☐	☐	☐	☐	☐	zwischen 07:30 - 08:00 Uhr abgeben, Personen vor Ort

Figure 2: Manual delivery schedule for the "City Tour" (ZH2 Stadt) showing customer addresses, opening times, departure times for the vehicle, and logistical remarks including drop-off locations and strict time windows.

4.2 Data Quality Assessment

Reviewing the data revealed several issues that required extensive preprocessing before the data could be used for modeling. These issues fell into three categories: structural, inconsistent, and incomplete data anomalies.

4.2.1 Structural and Formatting Issues

The greatest challenge was the lack of structure in the raw data:

- Fragmentation: The two workbooks made it difficult to understand the total demand for any specific day of delivery, eg. Monday.
- Combined Fields: Key fields—such as Name and Address, and Opening Time and TK indicators—were fused into single cells, preventing direct parsing. This can be witnessed in Figure 1.
- Single-Timeline Problem: A single “Departure Time” column was used for the entire week, even though daily routes differ. This can be seen in the Figure 1 for Customer Index 267 where they are visited on Monday but not on any other day, making the timeline unreliable for dynamic and daily routing.

4.2.2 Incompleteness and Inconsistency

- Incomplete Addresses: Some entries lacked building numbers, postal codes, or city names. One contained only a customer name and no address. These instances can be witnessed in Figure 3 below for Customer Index 247, and the final row in the Figure 3.

247	Hiltl, Dachterrasse, Zürich
359	apoTHEKE Züribergstrasse 17, Zürich
354	Theater Rigiblick, Germaniastasse 99, 8044 Zürich
258	Tibits Oerlikon, Tramstrasse 2, Zürich
501	Novu Campus AG, The Circle 61, Anlieferung B1, Flughafen Kloten
421	biolocal Flughafen, Tor 101, Flughafen Kloten
525	Gate Gourmet Switzerland, Borddienstrasse, 8058 Zürich
101	Kornhaus, Sonnenhofstrasse 2, 9500 Wil
	Leergut Markt Depot Altstadt

Figure 3: Raw data excerpt highlighting quality issues, such as incomplete address details (Index 247) and purely name labels without address (final row).

- Inconsistent Delivery Day Markings: While most deliveries were marked with “☺”, others used “XXX” or “(☺)”, creating ambiguity. This can be seen in Figure 4 below for Customer Index 499, 381, and 479.

499	Weber Comestibles, Forchstrasse 199, 8125 Zollikerberg	10:00	9:15	XXX	☺	XXX	XXX	☺	XXX
15 Min. Pause vor TK Kunden									
444	Stöckenweid Feldmeilen, Bünishoferstrasse 295,	TK		XXX		☺			
414	Stiftung Stöckenweid, Haus Seeblick, Eggrüti 4, Feldbach	TK		XXX		☺			
381	Café Fortuna, Dorfstrasse 12, 9657 Unterwasser	TK		XXX		☺		XXX	
479	Alnatura Stäfa, Bahnhofstrasse 8712 Stäfa	Spez				(☺)			
479	Alnatura Uster, Zürcherstrasse 14, 8610 Uster	Spez				(☺)			
389	Alnatura Thalwil, Alte Landstrasse, 8800 Thalwil	Spez		XXX		XXX			

Figure 4: Excerpt illustrating data inconsistencies, including ambiguous markers (“XXX”, “(☺)”) and “TK” entries that lack explicit time windows despite requiring strict handling.

- Missing Time Windows: Many customers marked “TK” needed strict handling but had no explicit delivery time information. This can also be observed in Figure 4 above for Customer Index 444, 471.
- Obscure Constraints: Important instructions—such as alternative delivery locations or customer-specific time windows—were buried in the free-text Remarks column. This is observed for Customer Index 503, 505 in Figure 5 and Customer Index 428 and 524 in Figure 6.

417	PaRadiesLi, Lederbachweg 1, 9620 Lichtensteig	9:00	4:50			☺	☺		In die Garage stellen > vis à vis Geschäft
503	MAJO, Hauptgasse 12, 9620 Lichtensteig		4:57						bei PaRadiesLi in die Garage stellen
178	Bio&Fair, Volkshausstrasse 22, 9630 Wattwil	9:00	4:57			☺	☺	☺	Beim Eingang Ladentisch
399	APODRO Reform Jona, Molkereistrasse 16, Jona	8:30	5:25			☺			grosser Schlüssel für mittleres Schloss
379	Ortimo AG, obere Bahnhofstrasse 58, Rapperswil	8:30	5:30	☺	☺	☺	☺	☺	geht vorne und hinten
505	Naturlade Wald GmbH, Schlipfstrasse 12, 8636 Wald		5:30						Bei Ortimo abladen

Figure 5: Excerpt showing critical logistical dependencies obscured in free-text remarks, specifically alternative drop-off locations.

434	Vita Volta, Voltastrasse 40, Fluntern Zürich	7:00	6:50	☺	☺	☺	☺	☺	Darf nicht vor 06:50 Uhr geliefert werden > im Laden abgeben
310	L'Ultimo Bacio, Nordstrasse 227, Wipkingen	7:00	7:00	☺	☺	☺	☺	☺	Im Laden abgeben/Retouren auf Terrasse
497	Chez Nous ZH GmbH, Röschlichbachstrasse 79, Zürich	TK	7:05	☺	☺	☺	☺	☺	Auf Abruf ca. 1x im Monat
415	Sphères, Hardturmstrasse 66, Zürich	8:00	7:10	☺	☺	☺	☺	☺	Ins Restaurant abgeben. Falls Restaurant geschlossen auf einer leeren Kiste in Gang stellen > bei geschlossenem Restaurant > ALARM
427	Tritt-Käse, Limmatstrasse 231, Viadukt, Zürich	8:00	7:15	☺	☺	☺	☺	☺	In Laden stellen
428	Südhang Weine, Limmatstrasse 231, Viadukt, Zürich	8:00	7:15		☺				Bei Tritt-Käse separat abgeben
469	Onno GmbH, Nordstrasse 89, 8037 Zürich	10:00	7:20	☺	☺	☺	☺	☺	Bei Café des Amis abgeben > separater Stapel
524	Café des Amis, Nordstrasse 88, 8037 Zürich	8:00	7:20	☺	☺	☺	☺	☺	zwischen 07:30 - 08:00 Uhr abgeben, Personen vor Ort

Figure 6: Excerpt highlighting unstructured constraints, where specific delivery windows are embedded in the remarks field rather than structured columns.

4.3 Data Cleaning and Preprocessing Strategies

To prepare the dataset for the solver, a structured multi-step cleaning process was implemented.

4.3.1 Restructuring and Standardization

1. Consolidation: The Lake Tour and City Tour spreadsheets were merged into one unified dataset. For comparison later, the Lake and City labels for each customer were kept in a separate column.
2. Field Separation: The combined “Customer Details” field was split into Name, Street, Building Number, City, and Postal Code. The “Service Requirements” field was also separated into Opening Times and the TK indicator.

4.3.2 Enrichment and Validation

1. Address Verification: Incomplete or inconsistent addresses were corrected manually using Google Maps. The customer with no address was removed after the bakery confirmed they belonged on a different, out-of-scope route.
2. Handling Ambiguous Labels: Ambiguous delivery-day symbols (“XXX”, “(☺)”) were treated as non-delivery days. Only explicit “☺” entries were accepted.
3. Service Time Inference for Frozen Goods: Customer marked as “TK” that lacked explicit service times were inferred manually using Google Maps and the customer’s position in the existing tour sequence, assuming that the existing sequence was operationally valid.
4. Remarks Extraction: Specific time windows mentioned in remarks were moved to a dedicated time window column, and delivery locations differing from the main address were highlighted using a different color for the geocoding phase.

4.3.3 Iterative Geocoding Correction

During geocoding, the goal was to resolve every address into a unique Google Place ID rather than simple coordinates. A number of customers initially failed to resolve due to errors, such as incorrect building numbers. These triggered an iterative correction process, where the problematic address was manually searched, cleaned, and re-submitted. This ensured that every customer in the final dataset had a valid, precise Place ID required for accurate optimization.

Figure 7 below shows the changes discussed in section 4.3.

Lake or City Tour	Remarks for Delivery	Company Index	Time Constraints	Opening Times	Frozen Food	Abfahrt	Name	Street/Area	Building Number	City	Postal Code	Monday
City	On demand approximately once a month Drop off at the restaurant. If the restaurant is closed, place it on an empty crate. If the restaurant is closed,	437	4:20 to 10:00	9:00	TK	7:05	Chez Nous ZH GmbH	Röschlibachstrasse	79	Zürich	8037	☉
		415										
City	ALARM.		4:20 to 8:00	8:00	-	7:10	Sphéres	Hardturmstrasse	66	Zürich	8005	☉
City	Place in store Drop off at Café des Amis >	427	4:20 to 8:00	8:00	-	7:15	Trit-Käse	Limmatstrasse	231	Zürich (Viadukt)	8005	☉
City	separate stack between 07:30 - 08:00, people on site	469	7:30 to 8:00	10:00	-	7:20	Onno GmbH	Nordstrasse	89	Zürich	8037	☉
City	Someone will be on site from 8:00 a.m. at the earliest. Parking at the bridge, about 400m walk.	524	7:30 to 8:00	8:00	-	7:20	Café des Amis	Nordstrasse	88	Zürich	8037	☉
		516					Letten Badi (Tel 079 704 64 52 Joachim)	Lettensteg	10	Zürich	8005	☉
City			8:00 to 9:00	9:00	-	8:10						☉

Figure 7: Processed dataset following restructuring and enrichment.

The table illustrates the separation of address components, standardization of time constraints, and the extraction of critical delivery rules from free-text remarks (highlighted in orange).

4.4 Geocoding and Place ID Extraction

After cleaning the dataset, all customer addresses were verified and available in text form. To build a reliable proof-of-concept for the optimization model, the next steps—geocoding and route solving—were carried out only on customers with Monday deliveries. Focusing on a single day made it easier to test and confirm that the solver worked correctly, while still providing proof that the setup that can later be expanded to the other weekdays.

For the Monday subset, the addresses first had to be converted into exact geographic data so that travel times could be calculated. To do this, a Python script was created that sends each address to the Google Maps Geocoding API. The script returns the coordinates (latitude and longitude) as well as the Google Place IDs, which are needed for consistent routing results.

The full geocoding script is included in [Appendix A](#). In this prototype, the Monday addresses were entered directly into the script as a fixed list. This provided a controlled environment to test the API connection and data extraction without dealing with file-handling issues. In complete implementation, this hardcoded list would be replaced with an automated import function that reads the addresses from the main CSV file or Excel file.

4.4.1 Request Logic and Data Extraction

The script uses a function called `geocode_address`, which iteratively processes every clean address. For each address, it sends a request to the Google Geocoding API.

When the API returns a successful response (status = OK), the script extracts key pieces of information:

- Formatted Address – Used to verify that the API understood the address correctly.
- Latitude and Longitude – The exact coordinates of the location.
- Place ID – A unique identifier in Google's database. Unlike raw coordinates, Place IDs consistently refer to the same building or business and ensure that later calculations (such as the Duration Matrix) reference the correct location.

```
# Queries Google Maps Geocoding API. Returns a dictionary with coordinates or empty fields on failure.
def geocode_address(address, api_key):
    endpoint = "https://maps.googleapis.com/maps/api/geocode/json"
    params = {
        'address': address,
        'key': api_key
    }

    try:
        response = requests.get(endpoint, params=params)
        response.raise_for_status() # halting on 4xx/5xx errors
        data = response.json()

        # Checking the internal 'status'
        if data['status'] == 'OK':
            result = data['results'][0]

            if len(data['results']) > 1:
                print(f"Warning: Ambiguous address '{address}'. Defaulting to first result.")

            return {
                "original_query": address,
                "formatted_address": result.get('formatted_address'),
                "place_id": result.get('place_id'),
                "latitude": result.get('geometry', {}).get('location', {}).get('lat'),
                "longitude": result.get('geometry', {}).get('location', {}).get('lng'),
                "status": data['status'],
                "result_count": len(data['results'])
            }
    }
```

Figure 8: Python implementation of the `geocode_address` function (Part 1). The script constructs the parameterized API request and processes successful responses (status: OK) to extract the specific Google Place ID and geographic coordinates.

4.4.2 Ambiguity Handling

Sometimes the API returns multiple possible matches for a single address and the script detects this through `len(results) > 1`. When this happens, the script issues a warning and automatically selects the first result, which Google ranks as the most likely match. The warning marks the entry for manual review to confirm it is the correct delivery address.

4.4.3 Error Handling and Status Validations

To make the process stable, the script includes multiple safeguards:

- Connection errors: Wrapped in a try/except block so the script doesn't stop if the internet connection fails temporarily.
- API status errors: If the API returns `ZERO_RESULTS`, `REQUEST_DENIED`, or `OVER_QUERY_LIMIT`, the script does not crash. Instead, it outputs an empty record, marking the address for later correction.

```
# Handling ZERO_RESULTS, REQUEST_DENIED, or OVER_QUERY_LIMIT
else:
    return {
        "original_query": address,
        "formatted_address": None,
        "place_id": None,
        "latitude": None,
        "longitude": None,
        "status": data['status'],
        "result_count": 0
    }

except Exception as e:
    print(f"Exception during request for '{address}': {e}")
    return {
        "original_query": address,
        "status": "ERROR",
        "error_message": str(e)
    }
```

Figure 9: Python implementation of the `geocode_address` function (Part 2). The logic handles non-successful API statuses (e.g., `ZERO_RESULTS`) and includes a try/except block to ensure pipeline resilience against network exceptions.

4.4.4 Data Storage and Export

After geocoding, the script saves the results in two formats:

- JSON (`geocoded_locations.json`): Keeps the full data structure and metadata. This file can be used directly by the Duration Matrix script.

- CSV (geocoded_locations.csv): A simpler, spreadsheet-friendly version created with pandas. It allows a final manual check to ensure the "Original Query" matches the "Formatted Address" before running the optimization.

4.5 Duration Matrix Calculation

With each customer mapped to a unique Google Place ID, the next step was to calculate the travel time between every pair of locations. Because this VRPTW model calculates time rather than distance, a complete, $N \times N$, Duration Matrix (where, N is the number of stops) was required. A Python script was developed to generate this matrix. It is important to note that the Google Maps API called on in this script provides both distance and duration matrixes and they both are saved but the process described below focuses on the duration matrix. The complete python script can be found in [Appendix C](#).

4.5.1 API Choice and Traffic Prediction

The script uses the Google Routes API (v2), specifically the **computeRouteMatrix** endpoint. This API was chosen instead of the standard Distance Matrix API because it provides more accurate travel-time estimates through predictive traffic modeling.

The request includes two key settings:

- Routing Preference: **TRAFFIC_AWARE_OPTIMAL**: Ensures routes are based on real-time and historical traffic data instead of simple road speeds. This can be seen in Figure 11.
- Predictive Departure Time: The script automatically sets the departure to “tomorrow at 07:00 AM.” This allows Google to use historical data to estimate typical Monday morning traffic, making the model reflect realistic delays rather than ideal conditions. This can be seen in Figure 10 below.

```
print(f"\nPreparing to build a {num_loc}x{num_loc} matrix using {API_CHUNK_SIZE}x{API_CHUNK_SIZE} chunks.")

now = datetime.datetime.now(datetime.timezone.utc)
tomorrow = now + datetime.timedelta(days=1)
departure_time = tomorrow.replace(hour=8, minute=0, second=0, microsecond=0)
departure_time_str = departure_time.isoformat().replace("+00:00", "Z")
print(f"Using predictive departure time: {departure_time_str}")

api_session = setup_session_with_retries()
```

Figure 10: Python logic for calculating a predictive departure time. To ensure realistic traffic estimates for the Monday route, the script dynamically sets the API timestamp to the next day at 08:00 AM

4.5.2 Batching and Matrix Construction

Since the full matrix contains N^2 travel-time entries, requesting everything at once would exceed API limits. To avoid this, the script builds the matrix in smaller 10×10 batches.

For each batch:

- A group of 10 origins is paired with 10 destinations.
- The script sends a JSON request for this block.
- The API returns the travel times in seconds.
- The values are inserted into the correct position in the global matrix.

The diagonal (travel from a location to itself) is manually set to 0.

4.5.3 Fault Tolerance and Checkpointing

Because building the matrix can take several minutes and involves many API calls, the script includes a checkpointing system:

- After each successful batch, the current matrix is saved to a `.npy` file.
- When restarted, the script loads the existing matrix and skips any entries already filled.

This makes the entire process resilient to crashes, timeouts, or internet interruptions and prevents wasted API calls.

4.5.4 Final Output

Once all batches are processed, the script exports the completed Duration Matrix as **Monday_Customers_duration_matrix.csv**. This file becomes the main numerical input for the OR-Tools VRPTW model.

```

# Loop through the matrix in chunks
for i in range(0, num_loc, API_CHUNK_SIZE):
    origin_start = i
    origin_end = min(i + API_CHUNK_SIZE, num_loc)
    origin_chunk = locations[origin_start:origin_end]

    for j in range(0, num_loc, API_CHUNK_SIZE):
        dest_start = j
        dest_end = min(j + API_CHUNK_SIZE, num_loc)
        dest_chunk = locations[dest_start:dest_end]

        print(f"\n Processing Chunk ")
        print(f"Origins:      {origin_start} to {origin_end-1}")
        print(f"Destinations: {dest_start} to {dest_end-1}")

        # Resume logic: Check if the first element of this chunk is populated
        # Also check that it's not a diagonal chunk (which starts with 0)
        if origin_start != dest_start and distance_matrix[origin_start, dest_start] != 0:
            print(f" > Skipping chunk, data already populated from checkpoint.")
            continue

        body = {
            "origins": origin_chunk,
            "destinations": dest_chunk,
            "travelMode": "DRIVE",
            "routingPreference": "TRAFFIC_AWARE_OPTIMAL",
            "departureTime": departure_time_str
        }

        api_response = call_routes_api(api_session, body, API_KEY)

```

Figure 11: The core matrix construction loop. The script iterates through the dataset in 10 x 10 chunks to respect API limits. The resume logic implements a checkpoint recovery mechanism, skipping chunks that were successfully processed in a previous run.

4.6 Solving the VRPTW

After preparing the dataset and duration matrix, the optimization model was built using the `ortools.constraint_solver` library. Although the general structure follows Google's standard VRPTW template (Google, 2024-c), several parameters were adapted for the bakery's requirements.

4.6.1 Model Initialization

Two OR-Tools components form the foundation of the model:

- Routing Index Manager: Maps the matrix indices to the solver's internal representation.
- Routing Model: Stores all constraints, dimensions, and the objective function.

The model uses 2 vehicles, and Node 0 (the bakery) serves as the shared start and end point.

```
def main():

    data = create_data_model()

    # Create the routing index manager.
    manager = pywrapcp.RoutingIndexManager(
        len(data["time_matrix"]), data["num_vehicles"], data["depot"]
    )

    # Create Routing Model.
    routing = pywrapcp.RoutingModel(manager)
```

Figure 12: Code snippet showing the initialization of the `RoutingIndexManager` and `RoutingModel`, which establish the solver's internal node mapping and constraint environment.

4.6.2 Time Dimension and Constraints

The VRPTW relies heavily on the Time Dimension, which tracks accumulated travel across each route (Google, 2024-a). The model configures it with:

- Slack (waiting time) of 72,000 seconds: Allows trucks to wait at a location if they arrive before the opening time.
- Maximum route duration of 72,000 seconds: A generous upper bound to keep routes finite.
- Customer Time Windows: Each customer's required delivery window $A = \pi r^2$ is enforced strictly. Routes arriving before or after the window are rejected.

```
# Configure Time Dimension
time = "Time"
routing.AddDimension(
    transit_callback_index,
    72000, # allow waiting time
    72000, # maximum time per vehicle
    False, # Don't force start cumul to zero.
    time,
)
time_dimension = routing.GetDimensionOrDie(time)
```

Figure 13: Configuration of the Time Dimension using `AddDimension`

4.6.3 Dropping Customers (Penalty Mechanism)

To avoid situations where the solver finds **no feasible solution**, each customer is assigned a very large penalty of 1,000,000. This line in the script shows the adding of the penalty:

```
routing.AddDisjunction([manager.NodeToIndex(node)], penalty)
```

This indicates to the solver that it may skip this customer only if absolutely necessary, but that it is extremely costly to do so. This guarantees that the solver always returns a solution while still prioritizing serving all customers.

4.6.4 Search Strategy Configuration

Using the algorithm setup described in Section 3.4, the solver was manually configured to test different search strategies without changing the underlying model. The goal of this experimental phase was to find which combination of heuristics delivers the best performance for the bakery's specific requirements.

To evaluate the solver, several combinations of strategies were tested:

- Constructive heuristics: The initial solutions were generated using **Path Cheapest Arc, Global Cheapest Arc, Savings, and Parallel Cheapest Insertion**. These different starting points helped create a broad range of possible route configurations.
- Metaheuristics: To improve these initial solutions and avoid getting stuck in local optima, the solver applied Greedy Descent, Guided Local Search (GLS), Simulated Annealing (SA), and Tabu Search.

Time Limit Conditions:

All strategy combinations were tested under the same constraints to ensure fair comparison. Each run was executed with fixed time limits of 10, 20, 30, and 60 seconds. This allowed an assessment of how solution quality changes as more computation time is provided.

```
# Setting first solution heuristic.
search_parameters = pywrapcp.DefaultRoutingSearchParameters()
search_parameters.time_limit.seconds = 60
search_parameters.log_search = True
#First Search Strategy
search_parameters.first_solution_strategy = (
    routing_enums_pb2.FirstSolutionStrategy.GLOBAL_CHEAPEST_ARC
)
#Local Search Metaheuristic Strategy
search_parameters.local_search_metaheuristic = (
    routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
)
```

Figure 14: Solver configuration snippet.

The code illustrates the setup for one of the experimental runs, specifically utilizing Global Cheapest Arc for the initial solution and Guided Local Search (GLS) for optimization, with a runtime limit of 60 seconds.

4.6.5 Output Interpretation

The solver works in raw seconds, starting from a base time of 04:20 AM = 0. The `print_solution_with_slack` function converts the solver's output into readable timestamps. The final schedule lists each stop in order, along with the exact arrival time for each of the two delivery trucks.

4.7 Process Visualization and Workflow Summary

To summarize the steps described in this chapter, the entire solution was mapped using a Business Process Model and Notation (BPMN) diagram. This visual model helps show how data moves through the system, how the internal optimization engine interacts with external APIs, and where data cleaning logic is applied.

4.7.1 High-Level Solution Architecture

Figure 1 presents the overall workflow of the Vehicle Routing Process. The process begins with the bakery's raw delivery data and moves through preprocessing, geocoding, travel-time calculation, and finally the VRPTW solution.

Key components include:

Bakery Pool: Provides the initial customer data and receives the final optimized delivery plans.

Google API Pool: Handles geocoding (Place IDs) and the Duration Matrix calculations.

Sequential Flow:

The BPMN diagram shows the dependencies between steps.

- The solver cannot run until the Duration Matrix is created.
- The Duration Matrix cannot be created until all locations have valid Place IDs.

This visualization clarifies how each stage relies on the successful completion of the previous one.

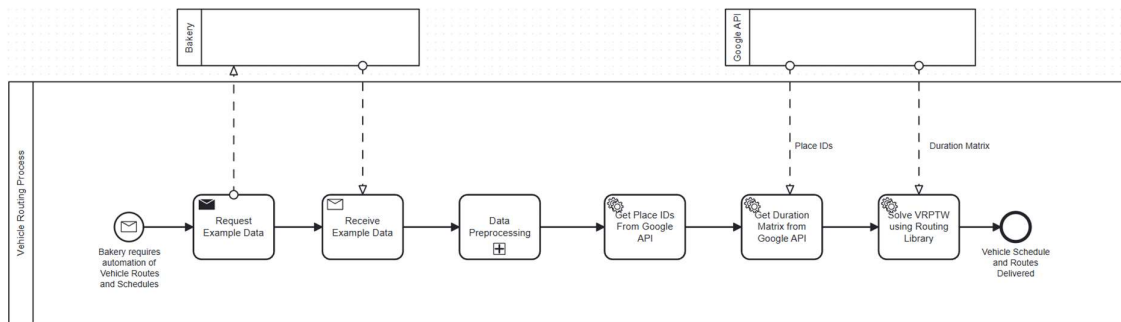


Figure 15: High-level BPMN diagram showing the interaction between bakery data, the Python processing pipeline, and Google Maps APIs.

4.7.2 Data Preprocessing Subprocess

The “Data Preprocessing” step (marked with a “+” in Figure 15) is expanded in Figure 16. This subprocess represents the detailed cleaning workflow described earlier in Sections 4.2 and 4.3.

The diagram highlights:

- Address verification: If an address is unclear or incomplete, a decision gateway triggers a feedback loop with the bakery until the location is confirmed.
- Time window extraction: Customer remarks containing special delivery constraints override standard delivery hours.

- Final output: Once all data is validated and cleaned, the customer list is split by delivery day and passed to the geocoding stage.

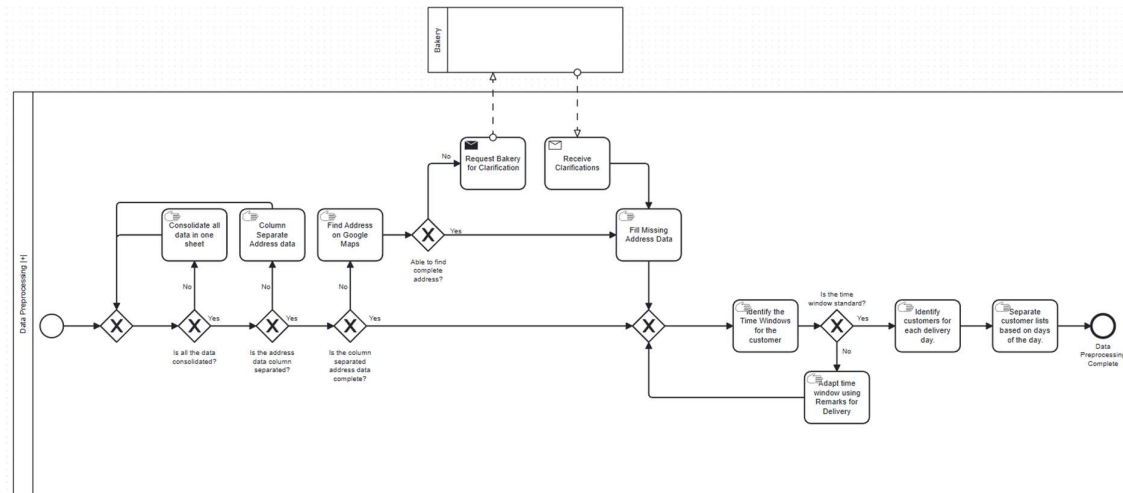


Figure 16: Detailed BPMN subprocess for Data Preprocessing. This flow illustrates the logic for address consolidation, the feedback loop for missing data, and the extraction of specific time windows from unstructured remarks.

5. Results

Following the methodology from Section 4 and highlighted in Section 3.3 the model was tested with a combination of Constructive Heuristics, Metaheuristics and Time limits. The goal was to find the best-performing configuration for the Lanterswil depot. The experiment combined:

- Five constructive heuristics (first solution strategies)
- Four metaheuristics (local search strategies as labeled by Google OR-Tools)
- Four runtime limits (10, 20, 30, and 60 seconds)

The objective function measures the total cost of the fleet's route. It minimizes two things: the travel time between stops and the penalties for any customer that cannot be served. However, it does not penalize waiting time. This means the solver prefers routes with the shortest driving distances, even if a driver arrives early and must wait for a time window to open. A lower objective value therefore indicates a more efficient travel schedule in terms of driving time and completed deliveries—not necessarily less idle time.

Appendix A showcases all the results of the tests. The objective values vary noticeably across strategies, ranging from a high of 26,144 (worst performance) to a low of 25,596 (best performance) and can be in the Table snippet below.

Table 1: A snippet of the worst and best performing configurations

Worst Performing			Best Performing		
Time Limit (s)	Constructive Heuristic	Christofides	Time Limit (s)	Constructive Heuristic	Global Cheapest Arc
	Metaheuristic			Metaheuristic	
60	Greedy Search	26144	60	Guided Local Search	26144 

5.1 Performance Analysis

5.1.1 Convergence at 25,629

A clear pattern appears across the tests: 25,629 is a strong local optimum.

- Savings Heuristic: The Savings method performed consistently well, reaching 25,629 even when paired with simple Greedy Search. This shows that the Clarke–Wright structure fits the Monday delivery map very well.
- GLS Correction: Guided Local Search (GLS) was able to “repair” weaker starting heuristics, such as Christofides or Parallel Cheapest Insertion, and push them down to the same 25,629 level within 10–20 seconds.

5.1.2 The Performance of Guided Local Search (GLS)

Among all the metaheuristics tested, Guided Local Search (GLS) consistently delivered the strongest improvements. In contrast, Simulated Annealing and Tabu Search frequently stalled and were unable to escape their initial high-cost solutions. GLS continued to make steady progress as the runtime increased, and at 30 seconds, the combination of GLS with the Global Cheapest Arc heuristic successfully broke below the 25,629 threshold, reaching a new global best of 25,596. Even at 60 seconds, GLS was able to further refine the Savings strategy to 25,621, though this result still fell short of the performance achieved with Global Cheapest Arc.

5.1.3 Best Configuration

The best overall setup for the Lanterwil Bakery is:

- First Solution Strategy: Global Cheapest Arc
- Metaheuristic: Guided Local Search
- Runtime: 30 seconds

This combination produced the lowest objective value (25,596) and represents the most efficient balance of travel time and time-window constraints for this heuristic and metaheuristic method of solution. Increasing the runtime to 60 seconds did not produce further gains, suggesting that 30 seconds is sufficient for this dataset.



5.2 Final Routes and Schedules

Table 2: Monday delivery routes and schedule for both delivery vans.

Vehicle 0 (Node)	Arrival Time (Hr:Min)	Wait Time (mins)		Vehicle 1 (Node)	Arrival Time (Hr:Min)	Wait Time (mins)
0 (Depot)	4:20	0		0 (Depot)	4:20	0
31	5:02	0		14	4:28	0
32	5:09	0		15	5:15	0
2	5:34	0		25	5:25	0
1	5:43	0		27	5:28	0
3	5:57	0		17	5:31	0
4	5:58	0		18	5:36	0
5	6:09	0		11	5:46	0
37	7:13	0		12	5:55	0
0(Depot)	7:25	0		13	6:04	0
				10	6:05	0
				9	6:08	0
				19	6:10	0
				35	6:13	0
				8	6:19	0
				20	6:25	0
				6	6:29	0
				7	6:32	0
				22	6:40	0
				21	6:41	0
				24	6:48	0
				23	6:49	0
				29	7:30	32
				26	7:32	0
				33	7:40	0
				28	7:45	0
				30	8:00	12
				34	8:05	0
				36	8:09	0
				16	8:20	0
				0 (Depot)	9:06	0

Based on the best-performing configuration (Global Cheapest Arc combined with Guided Local Search), the route and schedule plan for the Monday delivery fleet are shown above in Table 2.

5.3 Comparative Analysis: Algorithmic vs. Manual Planning

To evaluate the effectiveness of the solution, the schedule produced by the Guided Local Search metaheuristic was compared with the bakery's existing manually planned routes. The comparison focuses on two metrics: Total Tour Duration (the total time vehicles are active on the road) and Total Tour Distance (total kilometers driven).

5.3.1 Baseline Manual Schedule

The bakery currently uses a static routing approach in which both vehicles are planned to return to the depot at approximately 10:00 AM. This manual plan, shown in Table 3, relies primarily on intuitive geographic clustering and tries to fulfil all time-window constraints.

Table 3: Current Manual Delivery Plan (Baseline)

Vehicle 0 (Node)	Arrival Time (Hr:Min)		Vehicle 1 (Node)	Arrival Time (Hr:Min)
0 (Depot)	4:20		0 (Depot)	4:20
1	5:30		14	4:30
2	5:40		15	5:10
3	6:00		16	5:25
4	6:05		17	5:40
5	6:15		18	5:50
6	6:50		19	6:10
7	7:00		20	6:20
8	7:10		21	6:40
9	7:20		22	6:40
10	7:30		23	6:45
11	7:45		24	6:50
12	7:50		25	7:00
13	8:00		26	7:05
31	8:15		27	7:10
32	8:25		28	7:15
0(Depot)	10:00		29	7:20
			30	8:10
			33	8:25
			34	8:40
			35	8:42
			36	8:46
			37	9:45
			0(Depot)	10:00

5.3.2 Performance Metrics and Efficiency Gains

A quantitative comparison shows a substantial efficiency gap between the manual plan and algorithmic solution. As presented in Table 4, the manual plan requires a combined 11 hours and 20 minutes of active driving time. In contrast, the algorithmic solution reduces this to 7 hours and 51 minutes, representing a 30.74% improvement.

Table 4: Performance Comparison: Manual vs. Algorithmic Solution

Metric	Manual Plan	Algorithmic Solution	Difference
Total Tour Duration	11:20 (40,800 sec)	07:51 (28,260 sec)	-30.74% (Improvement)
Vehicle 0 Duration	5:40	3:05	
Vehicle 1 Duration	5:40	4:46	
Total Distance	357.7 km	380.5 km	+ 6.37% (Increase)
Vehicle 0 Distance	185.3 km	209.3 km	
Vehicle 1 Distance	172. km	171.2 km	

Interestingly, this reduction in total time occurs alongside a 6.37% increase in distance traveled, rising from 357.7 km to 380.5 km. This apparent contradiction reflects the solver's preference for time efficiency over geographic closeness. In several cases, the algorithm selects slightly longer routes to reduce waiting times and align more effectively with customer time windows even though waiting time is not penalized.

Overall, the fleet drives 22.8 km more, yet the bakery gains approximately 3.5 hours of saved labor and vehicle operation per day. This represents a substantial operational advantage for a two-vehicle MSME.

5.4 Operational and Algorithmic Outcomes

While the algorithmic schedule provides clear efficiency gains, its internal structure reveals operational characteristics that stem directly from the objective function design.

5.6.1 Analysis of Idle Time

The optimized schedule minimizes travel duration, not total time spent on the route. Waiting time is therefore unpenalized in the objective function. As a result, the solver often chooses fast routes that cause early arrivals, even when this leads to unavoidable idle periods.

This effect is visible in the schedule for Vehicle 1 in the algorithmic solution:

- Node 29: Arrival at approximately 06:58, but service cannot begin until 07:30, resulting in 32 minutes of waiting.
- Node 30: Arrival before the 08:00 time window, requiring 12 minutes of idle time.

These periods are **mathematically optimal under** the current objective function because avoiding them would require slower or longer detours. However, from an operational perspective, they represent inefficiencies in labor utilization, especially when compared to the manual plan, which implicitly spreads work to ensure both vehicles remain active until 10:00 AM.

5.6.2 Route Imbalance

Another difference between the two plans is the imbalance in workload distribution:

- Manual Plan: Both vehicles return at 10:00 AM, creating a perfectly balanced operational day.
- Optimized Plan:
 - Vehicle 0 finishes at 07:25 (a 3h 05m shift).
 - Vehicle 1 finishes at 09:06 (a 4h 46m shift).

While a human planner would naturally attempt to equalize driving time, the algorithm prioritizes minimizing total travel duration. Balancing the routes would require Vehicle 0 to take inefficient detours into areas naturally belonging to Vehicle 1's cluster, ultimately increasing the overall travel time and reducing the 30% efficiency gain achieved.

Thus, the imbalanced routes are not a flaw, but a direct outcome of minimizing total travel time rather than workload fairness—a tradeoff that is important for the bakery to consider in future operational decisions.

5.7 Hardware Comparison and Computational Environment

To confirm that the routing solution can realistically be deployed in a bakery environment, a performance comparison was carried out across two different computing setups. The same optimization configuration, Guided Local Search with a 30-second time limit, was run in a cloud environment (Google Colab) and on a regular consumer laptop running Windows 11. The goal was to see whether the algorithm requires specialized high-performance computing infrastructure, or if normal office hardware is sufficient.

Table 5: Hardware Specifications and Performance Benchmarks

Feature	Cloud Environment (Google Colab)	Local Environment (Laptop)
Operating System	Linux 6.6.105+	Windows 11
Processor (CPU)	Intel Xeon @ 2.20GHz (x86_64)	AMD Ryzen 5 5500U with Radeon Graphics
Memory	12.67 GB	7.35 (8 GB Physical)
Time Limit	30.00 Seconds	30.00 Seconds
Final Objective Value	25,596	25,596
Result	Best Known Solution	Best Known Solution

The comparison shows that the algorithm runs efficiently on both systems. Even though the cloud instance uses a server-grade Intel Xeon processor with more memory, and the local laptop uses a consumer-grade AMD Ryzen with 8 GB of RAM, both reached the same best-known solution (25,596) within the 30-second time limit.

This demonstrates that for a problem size of around 40 nodes, the search space is manageable enough that standard office hardware is not a limiting factor. The solver was able to explore neighborhoods, apply Guided Local Search, and escape local optima just as effectively on the local laptop as on the cloud environment. This confirms that no specialized hardware or HPC resources are needed for daily use.

6.0 Budget Considerations

6.1 Operational Demand Analysis

To estimate the cost of running an algorithmic routing system, we first need to understand how many customers must be serviced each day. Although the bakery serves 93 unique customers, not all of them receive a delivery every day. Daily demand is much lower and varies across the week.

Figure 3 shows the number of confirmed deliveries per day, ranging from 39 customers on Monday to 45 customers on Tuesday and Friday. This daily subset, not the full customer list, is what determines the size of each run. For the cost estimates that follow, it is assumed that this weekly pattern remains consistent excluding the seasonal variations or holidays.

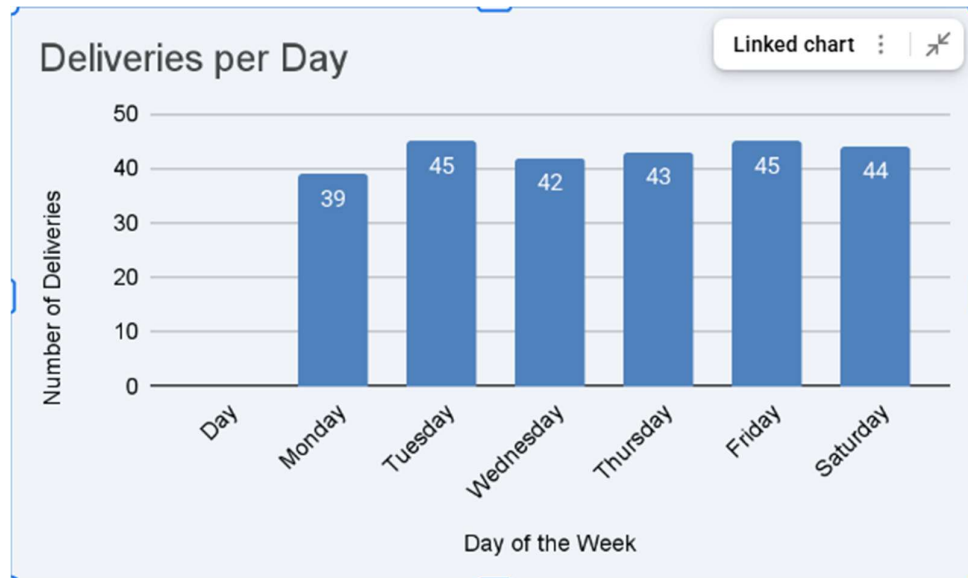


Figure 17: Distribution of confirmed deliveries per day of the week.

While the total unique customer base is 93, the daily optimization load is determined by these active subsets.

6.2

Component Cost Structure

The budget considers two parts: **Variable API Costs** (mainly Google Maps) and **Fixed Costs** (infrastructure and labour). While both are necessary, only Variable Costs are quantified. Further information from the bakery would be required to come an accurate conclusion for the Fixed Costs and is out of scope for this thesis. All numbers are in represented in CHF.

6.2.1 Variable API Costs (Google Maps Platform)

Geocoding API (Essentials Tier): This API converts addresses into Place IDs. It costs about 4.50 CHF per 1,000 requests. The bakery needs only 94 requests (93 customers + 1 depot). Even if the customer list doubled, usage would remain far below Google's free credit (about 180 CHF) even if the whole customer dataset increase 100 time. The Geocoding API effectively costs 0 CHF per month.

Routes API (Compute Route Matrix): This API generates travel-time matrices using the **TRAFFIC_AWARE_OPTIMAL** mode, which is part of the Pro Tier. It costs **9 CHF per 1,000 matrix elements**.

Matrix pricing depends on the number of elements. For a square matrix:

$$\text{Elements} = (N + 1)^2$$

where N is the number of customers and +1 accounts for the depot. Because the algorithm needs travel times between all pairs of points, each run requests a full square matrix.

6.2.2 Fixed Infrastructure and Operational Labor

Beyond API fees, the system requires basic operational support.

- **IT Infrastructure:** The Python script can run on existing office computers or a low-cost server. Hardware costs are minimal.
- **Operational Labor:** Since the system is not fully automated yet, a staff member needs to export orders, clean the CSV input, run the scripts, and read the output. This manual “human-in-the-loop” work takes time each evening.
- **System Maintenance:** Occasional technical work can be required, such as handling API keys or updating the script when the bakery’s operations change.

6.3 Operational Scenarios

To understand possible monthly costs, three operational setups were considered. Each uses a different number of API calls.

- **Scenario A:** One calculation per day (nightly).
- **Scenario B:** Nightly calculation + optional re-calculations when disruptions occur.
- **Scenario C:** Automatic re-calculation after every stop (real-time).

6.3.1 Scenario A: Nightly Static Solution

In this approach, the bakery calculates the route once per night based on predictive traffic data. For each day, a fresh distance matrix is produced based on that day’s active customer list. This scenario delivers a reliable plan at a manageable cost.

Example: Monday Deliveries

39 customers + 1 depot → 40 nodes

Matrix size: $40^2 = 1,600$ elements

Repeating this each day of delivery assuming the weekly demand from Figure 17 stays constant it leads to a Monthly (~4.3 weeks per month) cost estimate of: **~450.50 CHF**

6.3.2 Scenario B: Ad-Hoc On-Demand Solution

In this scenario, the nightly run from Scenario A is still used, but drivers or route manager may trigger a re-calculation if unexpected delays occur. This only happens in exceptional cases, such as traffic jams or road closures that could not have been factored in before.

Example: Assuming that the one vehicle driver is met with an unexpected delay such as a road closure due an accident halfway through their Monday route and requests for a renewal of the current route.

- Base nightly run: 14.40 CHF (for Monday).
- Singular mid-route delay leads to a new matrix for the remaining 20 stops:
 $20^2 = 400$ elements $\rightarrow$ **3.60 CHF**.

If such disruptions happen around twice a week, the monthly cost increases only marginally.

Estimated Monthly Cost: ~485 CHF

This adds flexibility while keeping expenses close to the Scenario A baseline.

6.3.3 Scenario C: Real-Time Dynamic Solution

This scenario re-calculates the optimal route after every single delivery stop to simulate a real-time dynamic solution that recalculates for even the slightest of changes. Because matrix sizes decrease by one node each time, the total number of elements is the sum of squares:

$$40^2 + 39^2 + 38^2 + \dots + 1^2$$

For Monday alone, this is about 22,140 elements, which costs 199.26 CHF for a single day.

Estimated Monthly Cost: ~6,800 CHF

This is far more expensive than the other scenarios and offers only marginal real-world benefits for a bakery with a predictable delivery area.

6.4 Operational Solution Recommendation

The budget analysis shows three very different cost levels:

- **Scenario A (~450 CHF/month):** The most balanced option. It gives reliable optimized routes at a sustainable cost.
- **Scenario B (~485 CHF/month):** This is the recommended strategy. It uses the nightly plan but also allows drivers to trigger a re-optimization during emergencies. The extra cost is small, and it offers resilience when disruptions occur.
- **Scenario C (~6,800 CHF/month):** Although highly accurate, it is over 1,400% more expensive than the baseline and unnecessary for the bakery's stable delivery environment.

7. Evaluation and Future Work

7.1 Critical Evaluation of Research Questions

This study set out to replace a manual, static routing process with an algorithmic solution tailored for an MSME. The success of this implementation is evaluated against the three research questions defined in the introduction.

7.1.1 RQ1: Solution Quality and Computational Efficiency

RQ1: Can the search algorithms provided by Google OR-Tools find correct solutions of good quality in reasonable time?

The results confirm that the heuristic approach is highly effective. The combination of the Global Cheapest Arc constructive heuristic and the Guided Local Search (GLS) metaheuristic generated a valid schedule in just 30 seconds.

- **Quality:** The algorithmic solution reduced total fleet active time by **30.74%** compared to the manual baseline, saving approximately 3.5 operational hours per day.
- **Trade-offs:** The solver prioritized time efficiency over geographic proximity, resulting in a 6.37% increase in total distance. This confirms that the algorithm correctly identified faster routes (likely highways or uncongested arterials) that were physically longer but temporally shorter than the manual "direct" routes.

- Conclusion: The solver successfully navigates the trade-off between strict time windows and travel duration, far exceeding the efficiency of manual planning.

7.1.2 RQ2: Scalability to Demand

RQ2: *Is the solution scalable to current and future demand?*

The scalability was tested across both software and hardware dimensions:

- Algorithmic Scalability: The "Monday" proof of concept included 40 nodes. Since the search strategies generally handle hundreds of nodes efficiently any near future scales of the bakery's customer base would remain well within the solver's capabilities without requiring a change in methodology.
- Hardware Scalability: The benchmark comparison between a cloud environment (Intel Xeon) and a standard consumer laptop (AMD Ryzen) showed identical performance (Objective Value: 25,596) within the same timeframe.
- Conclusion: No specialized infrastructure currently is required. The solution is fully deployable on the bakery's existing office hardware.

7.1.3 RQ3: Budgetary Implications

RQ3: *What are the budgetary implications of maintaining this system?*

The cost analysis identified Scenario B (Nightly + Ad-hoc) as the optimal strategy.

- Cost Efficiency: At approximately 485 CHF per month, the system costs significantly less than the value of the 3.5 daily labour hours saved.
- Avoidance of Excess: Scenario C (Real-Time Dynamic), estimated at ~6,800 CHF/month, was deemed excessive at the current stage. The marginal gain of real-time updates does not justify a 1,400% cost increase for a stable delivery network.
- Conclusion: The solution is financially viable for an MSME, as long as API calls are managed via a static or hybrid scheduling strategy rather than a continuous real-time loop.

7.2 Operational Limitations

While the solution is mathematically superior to the manual plan, the evaluation highlights two operational quirks inherent to the objective function:

1. Unpenalized Idle Time: As noted in the results, Vehicle 1 experiences significant wait times (e.g., 32 minutes at Node 29). Because the objective function minimizes *travel* time rather than *total shift* time, the solver allows early arrivals if they facilitate a faster driving route. While mathematically optimal, this creates perceived labour inefficiency.
2. Workload Imbalance: The optimized routes result in shifts of 3h 05m and 4h 46m. Unlike a human planner who instinctively balances work, the solver exploits Vehicle 0's proximity to early clusters to minimize global cost, leaving Vehicle 1 with a longer route.

7.3 Future Work

To transition this prototype into a fully production-ready system, future development should focus on three areas: algorithmic refinement, system integration, and cost optimization.

7.3.1 Algorithmic Refinements

To address the operational limitations identified in Section 7.2, the model constraints should be tuned:

- Waiting Time Penalties: The objective function should be modified to include a penalty term for waiting time (Slack). This would force the solver to seek routes that align arrival times more closely with time windows, reducing driver idleness.
- Route Balancing: Implementation of a GlobalSpan constraint in the Time Dimension would force the difference between the longest and shortest route to stay within a specific threshold, ensuring a fairer distribution of labor between drivers.

7.3.2 System Integration and Automation

Currently, the solution requires a "human-in-the-loop" to export CSVs, run Python scripts, and interpret results. Future work should focus on:

- Pipeline Automation: Developing a script that automatically fetches orders from the bakery's ERP system and pushes the final schedule to driver smartphones via a simple mobile interface or Google Maps link.
- Address Validation UI: Replacing the manual cleaning of addresses with a front-end interface that prompts bakery staff to validate new customer addresses immediately upon entry, preventing bad data from reaching the optimization stage.

7.3.3 Cost Optimization via Caching

The current variable cost model relies on generating a full square matrix $A = \pi r^2$ every night via the Google Routes API. However, the distance and duration between two static customers (e.g., Customer A to Customer B) rarely change significantly.

- Implementation: A local database (Cache) could store previously calculated leg durations. The API would only be queried for new customers or when traffic patterns significantly shift.
- Impact: This would drastically reduce the monthly operating cost, potentially lowering the API fees from ~485 CHF/month to near zero for established routes, making the system even more accessible for MSMEs.

7.3.4 Benchmarking with Exact Solvers

Finally, while heuristics provide a practical balance of speed and quality, future research could implement the problem using an exact solver like Gurobi or CPLEX (MILP). Although likely too slow for daily use, an exact solution would provide a definitive "optimality gap" benchmark, allowing the bakery to quantify exactly how close the heuristic solution is to the theoretical mathematical limit.

8. Conclusion

This thesis examined the delivery challenges faced by Micro, Small, and Medium-sized Enterprises (MSMEs) that operate under strict time-window requirements. Using a Swiss bakery as the case example, the goal was to determine whether modern, open-source optimization tools could realistically replace the bakery's manual routing process, which had become inefficient as operations grew.

The bakery's delivery system was modelled as a Vehicle Routing Problem with Time Windows (VRPTW). A full Python-based workflow was built using Google OR-Tools to clean the data, convert addresses into coordinates, and generate optimized delivery routes. Several search strategies were tested during the experiments, and the combination of Global Cheapest Arc with Guided Local Search (GLS) consistently delivered the best results for the bakery's constraints.

The findings highlight a clear performance difference between manual planning and algorithmic routing. The optimized plan reduced the bakery's daily active driving and service time by 30.74%, saving roughly 3.5 working hours per day. Although the solution produced a slightly longer total driving distance (+6.37%), this was intentional: the solver favoured faster, high-quality roads to

ensure all customers were served within their time windows—an area where manual planning often failed.

The study also showed that such optimization is technically and financially feasible for small businesses. The system ran efficiently on standard consumer hardware without requiring high-performance computing. From a financial perspective, the estimated operational cost of about 485 CHF per month (Scenario B) is small compared to the labour savings and the increased reliability gained from algorithmic planning.

Overall, the research demonstrates that advanced routing optimization is no longer limited to large companies. With accessible tools like Google OR-Tools, MSMEs can move from manual scheduling to automated, data-driven routing. Adopting these tools allows small enterprises to solve current inefficiencies while also laying a scalable foundation for future growth.

9. References

- Bräysy, O., & Gendreau, M. (2005). Vehicle routing problem with time windows, Part I: Route construction and local search algorithms. *Transportation Science*, 39(1), 104–118.
<https://cepac.cheme.cmu.edu/pasi2011/library/cerda/braysy-gendreau-vrp-review.pdf>
- Dantzig, G. B., & Ramser, J. H. (1959). The truck dispatching problem. *Management Science*, 6(1), 80–91.
<https://andresjaquep.wordpress.com/wp-content/uploads/2008/10/2627477-clasico-dantzig.pdf>
- Desrochers, M., Lenstra, J. K., Savelsbergh, M. W. P., & Soumis, F. (1988). Vehicle routing with time windows: Optimization and approximation. In B. L. Golden & A. A. Assad (Eds.), *Vehicle routing: Methods and studies* (pp. 65–84). North-Holland.
https://www.researchgate.net/publication/241887110_Vehicle_routing_with_time_windows_Optimization_and_approximation
- Google. (2024-a). *Dimensions*. Google for Developers. Retrieved December 9, 2025, from <https://developers.google.com/optimization/routing/dimensions>
- Google. (2024-b). *Routing options*. Google for Developers. Retrieved December 9, 2025, from https://developers.google.com/optimization/routing/routing_options
- Google. (2024-c). *Vehicle routing problem with time windows*. Google for Developers. Retrieved December 9, 2025, from <https://developers.google.com/optimization/routing/vrptw>
- Google. (2025-a). *How to cite OR-Tools*. Google for Developers. Retrieved December 9, 2025, from <https://developers.google.com/optimization/support/cite>
- Pacheco, J., Alvarez, A., García, I., & Angel-Bello, F. (2012). Optimizing vehicle routes in a bakery company allowing flexibility in delivery dates. *Journal of the Operational Research Society*, 63(5), 569–581.
https://www.researchgate.net/publication/227470389_Optimizing_vehicle_routes_in_a_bakery_company_allowing_flexibility_in_delivery_dates
- Rossi, F., van Beek, P., & Walsh, T. (Eds.). (2006). *Handbook of constraint programming*. Elsevier.
https://www.dcs.gla.ac.uk/~pat/cpM/papers/CP_Handbook-20060315-final.pdf
- Saller, S., Koehler, J., & Karrenbauer, A. (2023). A systematic review of approximability results for traveling salesman problems leveraging the TSP-T3CO definition scheme. arXiv.
<https://arxiv.org/abs/2311.00604>

Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2), 254–265. [suspicious link removed]

Weisstein, E. W. (n.d.). *Traveling salesman problem*. Wolfram MathWorld. Retrieved December 9, 2025, from <https://mathworld.wolfram.com/TravelingSalesmanProblem.html>

Xu, S., Hu, X., He, W., & Li, K. (2023). *Analysis of various methodologies for the TSP problem*. ResearchGate.

[https://www.researchgate.net/publication/376829888 Analysis of various methodologies for the TSP problem](https://www.researchgate.net/publication/376829888_Analysis_of_various_methodologies_for_the_TSP_problem)

10. Appendix

Appendix A: Objective Function Values by Strategy and Runtime

Time Limit (s)	Constructive Heuristic	Path Cheapest Arc	Christofides	Parallel Cheapest Insertion	Savings	Global Cheapest Arc
	Metaheuristic					
10	Greedy Search	25684	26144	26127	25629	26021
	Guided Local Search	25629	25629	25629	25629	25629
	Simulated Annealing	25684	26041	26127	25629	26021
	Tabu Search	25684	25666	25648	25629	25668
20	Greedy Search	25684	26144	26127	25629	26021
	Guided Local Search	25629	25629	25629	25629	25629
	Simulated Annealing	25684	26041	26127	25629	26021
	Tabu Search	25684	25666	25648	25629	25668
30	Greedy Search	25684	26144	26127	25629	26021
	Guided Local Search	25629	25629	25629	25629	25596
	Simulated Annealing	25684	26041	26127	25629	26021
	Tabu Search	25684	25666	25648	25629	25668
60	Greedy Search	25684	26144	26127	25629	26021
	Guided Local Search	25629	25629	25629	25621	25596
	Simulated Annealing	25684	26041	26127	25629	26021
	Tabu Search	25668	25666	25648	25629	25668

Appendix B: Python Script for Geocoding and extracting customer Place Ids.

```

import pandas as pd

import requests

import json

from google.colab import userdata

try:
    API_KEY = userdata.get('GOOGLE_API_KEY')
    if not API_KEY:
        raise ValueError("API key is empty.")
except (userdata.SecretNotFoundError, ValueError) as e:
    print(f"Setup Error: {e}. Please ensure 'GOOGLE_API_KEY' is set in Colab Secrets.")
    API_KEY = None

addresses_to_geocode = [
    "Schulstrasse 8, Lanterwil 9503",
    "obere Bahnhofstrasse 58, Rapperswil 8640",
    "Schlipfstrasse 12, Wald 8636",
    "Gwattstrasse 7a, Pfäffikon SZ 8808",
    "Bahnhofwiese 11, Stäfa 8712",
    "Bahnhofstrasse 22, Stäfa 8712",
    "Schulhausstrasse 18, Meilen 8706",
    "Seefeldstrasse 29, Zürich 8008",
    "Falkenstrasse 12, Zürich 8008",
    "Stockerstrasse 38, Zürich 8002",
    "Badenerstrasse 171, Zürich 8004",
    "Bremgartenstrasse 18, Zürich 8003",
    "Altstetterstrasse 128, Zürich 8048",
    "Ginsterstrasse 1, Albisrieden 8047",
    "Birmensdorferstrasse 145, Zürich 8003",

```

"Hauptstrasse 29, Tobel 9555",
"Althoossteig 2, Zürich 8046",
"Saatlenstrasse 12, Zürich 8051",
"Hardturmstrasse 171, Zürich 8005",
"Aargauerstrasse 1, Zürich 8048",
"Zweierstrasse 56, Zürich 8004",
"Kirchgasse 4, Zürich 8001",
"Asylstrasse 70, Zürich 8032",
"Römerhofplatz 5, Zürich 8032",
"Toblerstrasse 51, Zürich 8044",
"Voltastrasse 40, Zürich 8044",
"Nordstrasse 227, Zürich 8037",
"Röschlibachstrasse 79, Zürich 8037",
"Hardturmstrasse 66, Zürich 8005",
"Limmatstrasse 231, Zürich 8005",
"Nordstrasse 89, Zürich 8037",
"Nordstrasse 88, Zürich 8037",
"Lettensteg 10, Zürich 8005",
"Mythenquai 95, Zürich 8002",
"Seestrasse 205, Kilchberg 8802",
"Hardstrasse 210, Zürich 8005",
"Halle Löwenstrasse, Zürich 8001",
"Freischützstrasse 1a, Zürich 8004",
"Bahnhofstrasse 88, Zürich 8001",
"Sonnenhofstrasse 2, Wil 9500"

]

geocoded_results = []

Queries Google Maps Geocoding API. Returns a dictionary with coordinates or empty fields on failure.

```
def geocode_address(address, api_key):

    endpoint = "https://maps.googleapis.com/maps/api/geocode/json"

    params = {
        'address': address,
        'key': api_key
    }

    try:

        response = requests.get(endpoint, params=params)
        response.raise_for_status() # halting on 4xx/5xx errors
        data = response.json()

        # Checking the internal 'status'
        if data['status'] == 'OK':
            result = data['results'][0]

            if len(data['results']) > 1:
                print(f"Warning: Ambiguous address '{address}'. Defaulting to first result.")

        return {
            "original_query": address,
            "formatted_address": result.get('formatted_address'),
            "place_id": result.get('place_id'),
            "latitude": result.get('geometry', {}).get('location', {}).get('lat'),
            "longitude": result.get('geometry', {}).get('location', {}).get('lng'),
            "status": data['status'],
            "result_count": len(data['results'])
        }
```



```
# Handling ZERO_RESULTS, REQUEST_DENIED, or OVER_QUERY_LIMIT
```

```
else:
```

```
    return {
        "original_query": address,
        "formatted_address": None,
        "place_id": None,
        "latitude": None,
        "longitude": None,
        "status": data['status'],
        "result_count": 0
    }
```

```
except Exception as e:
```

```
    print(f"Exception during request for '{address}': {e}")
```

```
    return {
        "original_query": address,
        "status": "ERROR",
        "error_message": str(e)
    }
```

```
if API_KEY:
```

```
    print(f"Processing {len(addresses_to_geocode)} addresses...\n")
```

```
    for address in addresses_to_geocode:
```

```
        print(f"Geocoding: '{address}'")
        result_data = geocode_address(address, API_KEY)
        geocoded_results.append(result_data)
```

```
    print("\n--- Processing Complete ---")
```

```
# Dumps the raw JSON to the console for verification
print(json.dumps(geocoded_results, indent=2))

# 1. Exports to JSON (preserves structure/metadata)
json_filename = 'geocoded_locations.json'
with open(json_filename, 'w') as f:
    json.dump(geocoded_results, f, indent=2)
print(f"Saved: {json_filename}")

# 2. Exports to CSV (flat format for tabular analysis)
csv_filename = 'geocoded_locations.csv'
try:
    df = pd.DataFrame(geocoded_results)
    df.to_csv(csv_filename, index=False)
    print(f"Saved: {csv_filename}")
except Exception as e:
    print(f"CSV Export Failed: {e}")

else:
    print("Aborting: API Key missing.")
```

Appendix C: Python Script for calculating the complete Duration Matrix for all locations including the depot on the route.

```
import pandas as pd
import numpy as np
import requests
import json
import datetime
import os
import time
from google.colab import userdata

from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

try:
    API_KEY = userdata.get('GOOGLE_API_KEY')
    if not API_KEY:
        raise ValueError("API key is empty. Please check your Colab Secrets.")
except userdata.SecretNotFoundError:
    print("Error: 'GOOGLE_API_KEY' secret not found.")
    API_KEY = None
except ValueError as e:
    print(e)
    API_KEY = None

CSV_INPUT_FILE = "Monday_Customers.csv"
PLACE_ID_COLUMN_NAME = "place_id"
API_URL = "https://routes.googleapis.com/distanceMatrix/v2:computeRouteMatrix"
```

```
# Output Files
```

```
OUTPUT_DISTANCE_MATRIX_FILE = "Monday_Customers_distance_matrix.csv"
```

```
OUTPUT_DURATION_MATRIX_FILE = "Monday_Customers_duration_matrix.csv"
```

```
# Checkpoint files
```

```
CHECKPOINT_DISTANCE_FILE = "distance_matrix.npy"
```

```
CHECKPOINT_DURATION_FILE = "duration_matrix.npy"
```

```
API_CHUNK_SIZE = 10
```

```
# Read the location data from the CSV file
```

```
def read_locations_from_csv(csv_file, place_id_col):
```

```
    try:
```

```
        df = pd.read_csv(csv_file)
```

```
    except FileNotFoundError:
```

```
        print(f"ERROR: CSV file not found at '{csv_file}'")
```

```
        return None, 0
```

```
    if place_id_col not in df.columns:
```

```
        print(f"ERROR: Column '{place_id_col}' not found in CSV.")
```

```
        return None, 0
```

```
    waypoints = [{"waypoint": {"placeId": str(pid)}} for pid in df[place_id_col]]
```

```
    num_locations = len(waypoints)
```

```
    print(f"Successfully read {num_locations} locations from '{csv_file}'.")
```

```
    print(f"Depot (Index 0) Place ID: {df[place_id_col].iloc[0]}")
```

```
    return waypoints, num_locations
```

```
# Create a requests.Session with automatic retries on server errors
```

```
def setup_session_with_retries():
```

```

session = requests.Session()

retries = Retry(total=3, backoff_factor=1, status_forcelist=[500, 502, 503, 504],
allowed_methods=["POST"])

session.mount("https://", HTTPAdapter(max_retries=retries))

return session

#Calls the Google Routes API
def call_routes_api(session, request_body, api_key):

    headers = {
        "Content-Type": "application/json",
        "X-Goog-API-Key": api_key,
        "X-Goog-FieldMask": "originIndex,destinationIndex,duration,distanceMeters,status"
    }

    num_origins = len(request_body['origins'])
    num_dests = len(request_body['destinations'])
    print(f" > Calling API for {num_origins}x{num_dests} sub-matrix...")

    try:
        response = session.post(API_URL, data=json.dumps(request_body), headers=headers)
        response.raise_for_status()
        response_data = response.json()
        print(f" > API call successful. Received {len(response_data)} matrix elements.")

        # 2-second pause
        print(" > Pausing for 2s to respect rate limit...")
        time.sleep(2)

        return response_data

    except requests.exceptions.HTTPError as http_err:

```

```

print(f"HTTP error occurred: {http_err}")
print(f"Response content: {response.text}")
if "API key not valid" in response.text:
    print("\n FAILED: CHECK API KEY ")
if "has not been used" in response.text:
    print("\n FAILED: ENABLE 'Routes API' IN GOOGLE CLOUD CONSOLE")
if response.status_code == 429:
    print("\n FAILED: 429 'Too Many Requests' / 'Resource Exhausted' ")
except requests.exceptions.RequestException as req_err:
    print(f" Error occurred (retries may have failed): {req_err}")

return None

#Parse the API response and update the main matrices
def populate_matrices_from_response(response_data, distance_matrix, duration_matrix,
origin_offset, dest_offset):

    elements_populated = 0
    for element in response_data:
        if 'originIndex' in element:
            local_origin_idx = element['originIndex']
            local_dest_idx = element['destinationIndex']

            # This check correctly handles self-to-self (e.g., [0,0])
            if local_origin_idx == local_dest_idx and origin_offset == dest_offset:
                distance_meters = 0
                duration_seconds = 0
            else:
                distance_meters = element.get('distanceMeters', 0)
                duration_str = element.get('duration', '0s')
                duration_seconds = int(duration_str.rstrip('s'))

```

```

    global_origin_idx = local_origin_idx + origin_offset
    global_dest_idx = local_dest_idx + dest_offset

    distance_matrix[global_origin_idx, global_dest_idx] = distance_meters
    duration_matrix[global_origin_idx, global_dest_idx] = duration_seconds
    elements_populated += 1
else:
    status_msg = element.get('status', {}).get('message', 'Unknown error')
    print(f" > WARNING: No route found for an element in this chunk. Status: {status_msg}")

print(f" > Populated {elements_populated} elements in the main matrices.")

#Save the matrices for checkpointing
def save_matrices_to_npy(distance_matrix, duration_matrix):

    np.save(CHECKPOINT_DISTANCE_FILE, distance_matrix)
    np.save(CHECKPOINT_DURATION_FILE, duration_matrix)
    print(" > Checkpoint saved.")

#Save the final matrices to CSV files
def save_matrices_to_csv(distance_matrix, duration_matrix):

    np.savetxt(OUTPUT_DISTANCE_MATRIX_FILE, distance_matrix, delimiter=',', fmt='%d')
    print(f"Distance matrix saved to '{OUTPUT_DISTANCE_MATRIX_FILE}'")
    np.savetxt(OUTPUT_DURATION_MATRIX_FILE, duration_matrix, delimiter=',', fmt='%d')
    print(f"Duration matrix saved to '{OUTPUT_DURATION_MATRIX_FILE}'")

def main():
    if not API_KEY:
        print(" ERROR: 'GOOGLE_API_KEY' secret not found or is empty. Exiting. ")

```

```

    return

locations, num_loc = read_locations_from_csv(CSV_INPUT_FILE, PLACE_ID_COLUMN_NAME)

if not locations:
    return

print(f"\nPreparing to build a {num_loc}x{num_loc} matrix using
{API_CHUNK_SIZE}x{API_CHUNK_SIZE} chunks.")

now = datetime.datetime.now(datetime.timezone.utc)
tomorrow = now + datetime.timedelta(days=1)
departure_time = tomorrow.replace(hour=8, minute=0, second=0, microsecond=0)
departure_time_str = departure_time.isoformat().replace("+00:00", "Z")
print(f"Using predictive departure time: {departure_time_str}")

api_session = setup_session_with_retries()

# Load from checkpoint or create new matrices
try:
    distance_matrix = np.load(CHECKPOINT_DISTANCE_FILE)
    duration_matrix = np.load(CHECKPOINT_DURATION_FILE)
    print(f" Loaded matrices from checkpoint files ")
    if distance_matrix.shape != (num_loc, num_loc):
        raise ValueError("Checkpoint shape mismatch. Starting fresh.")
except (FileNotFoundError, ValueError) as e:
    print(f" No valid checkpoint found ({e}). Creating new matrices. ")
    distance_matrix = np.zeros((num_loc, num_loc), dtype=int)
    duration_matrix = np.zeros((num_loc, num_loc), dtype=int)

# Loop through the matrix in chunks
for i in range(0, num_loc, API_CHUNK_SIZE):

```



```

origin_start = i
origin_end = min(i + API_CHUNK_SIZE, num_loc)
origin_chunk = locations[origin_start:origin_end]

for j in range(0, num_loc, API_CHUNK_SIZE):
    dest_start = j
    dest_end = min(j + API_CHUNK_SIZE, num_loc)
    dest_chunk = locations[dest_start:dest_end]

    print(f"\n Processing Chunk ")
    print(f"Origins: {origin_start} to {origin_end-1}")
    print(f"Destinations: {dest_start} to {dest_end-1}")

    # Resume logic: Check if the first element of this chunk is populated
    # Also check that it's not a diagonal chunk (which starts with 0)
    if origin_start != dest_start and distance_matrix[origin_start, dest_start] != 0:
        print(f" > Skipping chunk, data already populated from checkpoint.")
        continue

    body = {
        "origins": origin_chunk,
        "destinations": dest_chunk,
        "travelMode": "DRIVE",
        "routingPreference": "TRAFFIC_AWARE_OPTIMAL",
        "departureTime": departure_time_str
    }

    api_response = call_routes_api(api_session, body, API_KEY)

    if not api_response:

```

```

        print(f"ERROR: API call failed for chunk (Origins {origin_start}-{origin_end}, Dests
{dest_start}-{dest_end}).")

```

```

        print("Stopping script. Re-run to resume from this chunk.")

```

```

        return

```

```

    populate_matrices_from_response(api_response,

```

```

                                   distance_matrix,

```

```

                                   duration_matrix,

```

```

                                   origin_start,

```

```

                                   dest_start)

```

```

    # Save to checkpoint files after every successful chunk

```

```

    save_matrices_to_npy(distance_matrix, duration_matrix)

```

```

print("\n All chunks processed ")

```

```

save_matrices_to_csv(distance_matrix, duration_matrix)

```

```

print("\n SCRIPT FINISHED SUCCESSFULLY ")

```

```

if __name__ == "__main__":

```

```

    main()

```

Appendix D: Python Script for solving the VRPTW and getting the vehicle route and schedule.

```

from ortools.constraint_solver import routing_enums_pb2

from ortools.constraint_solver import pywrapcp

import pandas as pd

from datetime import datetime, timedelta

from IPython.display import display, HTML


#reads duration matrix from a csv file and creates data model

def create_data_model():

    data = {}

    raw_matrix = [

        [0.0, 3644.0, 3883.0, 4097.0, 4319.0, 4160.0, 3589.0, 3500.0, 3404.0, 3312.0, 3371.0, 3554.0,
        3325.0, 3455.0, 501.0, 2940.0, 2789.0, 3017.0, 3180.0, 3347.0, 3468.0, 3392.0, 3273.0, 3103.0,
        3104.0, 2953.0, 2975.0, 2906.0, 3067.0, 2960.0, 3034.0, 2574.0, 2968.0, 3114.0, 3129.0, 3327.0,
        3302.0, 702.0],

        [3577.0, 0.0, 395.0, 786.0, 727.0, 1197.0, 2355.0, 2396.0, 2041.0, 2083.0, 2046.0, 2697.0, 2472.0,
        2027.0, 3495.0, 2711.0, 2496.0, 2523.0, 2584.0, 2020.0, 2383.0, 2146.0, 2255.0, 2364.0, 2404.0,
        2436.0, 2463.0, 2542.0, 2393.0, 2494.0, 2417.0, 1839.0, 1478.0, 2410.0, 2386.0, 1980.0, 2260.0,
        3155.0],

        [3835.0, 563.0, 0.0, 1314.0, 1261.0, 1738.0, 2172.0, 2222.0, 1854.0, 1894.0, 1859.0, 2490.0,
        2282.0, 1830.0, 3773.0, 2532.0, 2619.0, 2332.0, 2386.0, 1830.0, 2174.0, 2365.0, 2450.0, 2539.0,
        2468.0, 2257.0, 2281.0, 2356.0, 2221.0, 2291.0, 2253.0, 1661.0, 1295.0, 2218.0, 2198.0, 1794.0,
        2071.0, 3399.0],

        [4142.0, 793.0, 1178.0, 0.0, 63.0, 639.0, 2067.0, 2135.0, 2276.0, 2668.0, 2635.0, 3244.0, 3026.0,
        2590.0, 3933.0, 2705.0, 2299.0, 2920.0, 3117.0, 2549.0, 2184.0, 1952.0, 2074.0, 2173.0, 2213.0,
        2634.0, 2627.0, 2803.0, 2716.0, 2668.0, 2622.0, 2335.0, 2244.0, 2970.0, 2511.0, 2502.0, 2637.0,
        3688.0],

        [4119.0, 825.0, 1188.0, 223.0, 0.0, 663.0, 2078.0, 2139.0, 2286.0, 2686.0, 2625.0, 3160.0, 3010.0,
        2601.0, 3999.0, 2795.0, 2383.0, 2915.0, 3111.0, 2530.0, 2180.0, 2026.0, 2146.0, 2268.0, 2291.0,
        2788.0, 2792.0, 2801.0, 2702.0, 2665.0, 2614.0, 2358.0, 2280.0, 2991.0, 2495.0, 2520.0, 2649.0,
        3720.0],

        [4065.0, 1314.0, 1673.0, 748.0, 648.0, 0.0, 1554.0, 1625.0, 1771.0, 2163.0, 2120.0, 2726.0, 2528.0,
        2090.0, 3783.0, 2575.0, 2170.0, 2389.0, 2617.0, 2036.0, 1666.0, 1661.0, 1766.0, 1911.0, 1959.0,
        2275.0, 2272.0, 2311.0, 2177.0, 2160.0, 2102.0, 1834.0, 2214.0, 2481.0, 1990.0, 1996.0, 2109.0,
        3854.0],
    ]

```

[3364.0, 2239.0, 2088.0, 1873.0, 1890.0, 1348.0, 0.0, 146.0, 507.0, 908.0, 870.0, 1486.0, 1296.0, 831.0, 3147.0, 1371.0, 1253.0, 1176.0, 1405.0, 725.0, 427.0, 392.0, 480.0, 684.0, 685.0, 1050.0, 1042.0, 1065.0, 966.0, 937.0, 852.0, 559.0, 965.0, 1174.0, 733.0, 729.0, 886.0, 3219.0],

[3238.0, 2064.0, 1954.0, 2004.0, 1977.0, 1445.0, 170.0, 0.0, 350.0, 707.0, 722.0, 1346.0, 1148.0, 679.0, 2983.0, 1224.0, 1102.0, 1039.0, 1243.0, 615.0, 249.0, 444.0, 515.0, 615.0, 543.0, 905.0, 893.0, 910.0, 810.0, 789.0, 699.0, 402.0, 807.0, 1038.0, 598.0, 545.0, 731.0, 3063.0],

[3184.0, 1831.0, 1705.0, 2120.0, 2022.0, 1472.0, 385.0, 439.0, 0.0, 374.0, 474.0, 1052.0, 830.0, 419.0, 2922.0, 1086.0, 951.0, 851.0, 924.0, 326.0, 375.0, 551.0, 633.0, 733.0, 659.0, 793.0, 812.0, 795.0, 670.0, 673.0, 590.0, 237.0, 647.0, 747.0, 539.0, 207.0, 413.0, 3005.0],

[3039.0, 1869.0, 1752.0, 2461.0, 2381.0, 1832.0, 758.0, 806.0, 389.0, 0.0, 267.0, 683.0, 469.0, 394.0, 2773.0, 748.0, 827.0, 527.0, 607.0, 158.0, 755.0, 867.0, 953.0, 951.0, 961.0, 448.0, 466.0, 548.0, 410.0, 539.0, 442.0, 560.0, 967.0, 413.0, 622.0, 196.0, 491.0, 2869.0],

[3120.0, 1776.0, 1645.0, 2409.0, 2350.0, 1805.0, 734.0, 796.0, 416.0, 159.0, 0.0, 769.0, 465.0, 236.0, 2868.0, 835.0, 911.0, 603.0, 682.0, 238.0, 741.0, 921.0, 998.0, 1098.0, 1030.0, 525.0, 548.0, 628.0, 489.0, 623.0, 522.0, 452.0, 859.0, 495.0, 701.0, 284.0, 579.0, 2948.0],

[3347.0, 2496.0, 2407.0, 3176.0, 3117.0, 2611.0, 1562.0, 1669.0, 1185.0, 891.0, 847.0, 0.0, 545.0, 999.0, 3184.0, 1064.0, 1246.0, 630.0, 468.0, 1016.0, 1566.0, 1563.0, 1660.0, 1368.0, 1368.0, 862.0, 888.0, 656.0, 814.0, 1051.0, 852.0, 1293.0, 1717.0, 693.0, 1193.0, 1038.0, 1248.0, 3222.0],

[3221.0, 2157.0, 2031.0, 2828.0, 2762.0, 2221.0, 1203.0, 1257.0, 884.0, 572.0, 435.0, 501.0, 0.0, 537.0, 3012.0, 1010.0, 1089.0, 731.0, 559.0, 657.0, 1205.0, 1363.0, 1450.0, 1195.0, 1214.0, 699.0, 731.0, 764.0, 661.0, 951.0, 697.0, 928.0, 1330.0, 674.0, 1031.0, 697.0, 985.0, 3097.0],

[3176.0, 1823.0, 1705.0, 2482.0, 2434.0, 1877.0, 825.0, 880.0, 508.0, 243.0, 88.0, 847.0, 436.0, 0.0, 2925.0, 913.0, 992.0, 685.0, 764.0, 332.0, 829.0, 1005.0, 1084.0, 1113.0, 1113.0, 605.0, 632.0, 709.0, 569.0, 708.0, 608.0, 539.0, 950.0, 578.0, 797.0, 360.0, 665.0, 3023.0],

[517.0, 3580.0, 3843.0, 3888.0, 3958.0, 3952.0, 3505.0, 3416.0, 3340.0, 3273.0, 3320.0, 3489.0, 3295.0, 3398.0, 0.0, 2811.0, 2570.0, 2904.0, 3068.0, 3278.0, 3367.0, 3322.0, 3038.0, 2908.0, 2918.0, 2848.0, 2870.0, 2784.0, 2950.0, 2849.0, 2926.0, 3520.0, 3887.0, 3027.0, 3038.0, 3302.0, 3244.0, 647.0],

[3031.0, 2776.0, 2694.0, 2609.0, 2671.0, 2545.0, 1511.0, 1378.0, 1356.0, 1244.0, 1048.0, 1163.0, 1236.0, 1349.0, 2759.0, 0.0, 784.0, 773.0, 863.0, 1328.0, 1359.0, 1160.0, 1082.0, 809.0, 816.0, 623.0, 646.0, 544.0, 698.0, 678.0, 731.0, 1562.0, 1942.0, 809.0, 924.0, 1249.0, 1187.0, 2829.0],

[2772.0, 2605.0, 2716.0, 2203.0, 2256.0, 2045.0, 1433.0, 1337.0, 1344.0, 1192.0, 1373.0, 1563.0, 1365.0, 1457.0, 2508.0, 739.0, 0.0, 893.0, 1079.0, 1235.0, 1289.0, 1105.0, 1110.0, 812.0, 817.0, 834.0, 860.0, 759.0, 894.0, 788.0, 865.0, 1488.0, 1871.0, 1028.0, 974.0, 1241.0, 1186.0, 2579.0],

[3001.0, 2427.0, 2319.0, 2626.0, 2687.0, 2346.0, 1316.0, 1234.0, 1096.0, 808.0, 875.0, 617.0, 693.0, 1042.0, 2785.0, 727.0, 801.0, 0.0, 300.0, 924.0, 1190.0, 1107.0, 1192.0, 943.0, 940.0, 438.0, 461.0, 140.0, 374.0, 586.0, 409.0, 1214.0, 1632.0, 304.0, 741.0, 947.0, 966.0, 2847.0],

[3023.0, 2396.0, 2273.0, 2641.0, 2689.0, 2485.0, 1404.0, 1275.0, 1080.0, 782.0, 853.0, 611.0, 643.0, 776.0, 2782.0, 757.0, 830.0, 222.0, 0.0, 899.0, 1236.0, 1187.0, 1224.0, 966.0, 971.0, 451.0, 472.0, 271.0, 412.0, 699.0, 447.0, 1175.0, 1573.0, 295.0, 774.0, 924.0, 973.0, 2866.0],

[3131.0, 1761.0, 1640.0, 2410.0, 2340.0, 1803.0, 727.0, 796.0, 398.0, 145.0, 158.0, 797.0, 523.0, 282.0, 2885.0, 861.0, 938.0, 634.0, 712.0, 0.0, 738.0, 915.0, 988.0, 1088.0, 1021.0, 549.0, 573.0, 656.0, 517.0, 527.0, 550.0, 444.0, 853.0, 525.0, 609.0, 178.0, 483.0, 2972.0],

[3210.0, 2036.0, 1895.0, 1996.0, 1914.0, 1362.0, 254.0, 307.0, 302.0, 653.0, 679.0, 1315.0, 1107.0, 636.0, 2970.0, 1206.0, 986.0, 1034.0, 1221.0, 565.0, 0.0, 437.0, 515.0, 613.0, 533.0, 891.0, 886.0, 898.0, 807.0, 782.0, 686.0, 360.0, 770.0, 1051.0, 591.0, 493.0, 589.0, 3039.0],

[3274.0, 2248.0, 2323.0, 1879.0, 1942.0, 1553.0, 541.0, 430.0, 758.0, 1022.0, 1145.0, 1636.0, 1443.0, 1108.0, 3059.0, 1261.0, 1147.0, 1110.0, 1284.0, 952.0, 591.0, 0.0, 232.0, 417.0, 434.0, 943.0, 941.0, 980.0, 895.0, 818.0, 773.0, 814.0, 1219.0, 1116.0, 665.0, 849.0, 880.0, 3141.0],

[3164.0, 2222.0, 2272.0, 1862.0, 1917.0, 1533.0, 506.0, 394.0, 701.0, 948.0, 1065.0, 1676.0, 1464.0, 1027.0, 2929.0, 1092.0, 966.0, 1042.0, 1219.0, 889.0, 542.0, 36.0, 0.0, 308.0, 334.0, 780.0, 775.0, 906.0, 819.0, 764.0, 706.0, 749.0, 1146.0, 1054.0, 607.0, 785.0, 823.0, 3012.0],

[2921.0, 2468.0, 2421.0, 2096.0, 2160.0, 1917.0, 852.0, 636.0, 849.0, 981.0, 1054.0, 1350.0, 1286.0, 1174.0, 2647.0, 804.0, 667.0, 819.0, 1015.0, 919.0, 607.0, 336.0, 353.0, 0.0, 101.0, 502.0, 498.0, 690.0, 637.0, 440.0, 625.0, 912.0, 1298.0, 942.0, 508.0, 807.0, 735.0, 2736.0],

[2943.0, 2501.0, 2391.0, 2136.0, 2191.0, 1878.0, 796.0, 674.0, 791.0, 930.0, 994.0, 1378.0, 1311.0, 1110.0, 2679.0, 821.0, 689.0, 832.0, 1030.0, 860.0, 535.0, 327.0, 351.0, 84.0, 0.0, 523.0, 517.0, 709.0, 645.0, 462.0, 576.0, 846.0, 1236.0, 928.0, 440.0, 757.0, 616.0, 2762.0],

[2763.0, 2359.0, 2288.0, 2331.0, 2384.0, 2134.0, 1125.0, 1018.0, 955.0, 654.0, 845.0, 860.0, 813.0, 936.0, 2491.0, 416.0, 489.0, 316.0, 510.0, 716.0, 984.0, 779.0, 807.0, 518.0, 526.0, 0.0, 24.0, 190.0, 347.0, 258.0, 324.0, 1154.0, 1564.0, 456.0, 563.0, 757.0, 791.0, 2579.0],

[2817.0, 2472.0, 2386.0, 2370.0, 2446.0, 2194.0, 1112.0, 1017.0, 942.0, 698.0, 780.0, 916.0, 903.0, 1030.0, 2554.0, 463.0, 539.0, 399.0, 594.0, 751.0, 937.0, 880.0, 952.0, 691.0, 689.0, 166.0, 0.0, 276.0, 365.0, 323.0, 390.0, 1090.0, 1478.0, 476.0, 552.0, 834.0, 774.0, 2623.0],

[2888.0, 2424.0, 2318.0, 2486.0, 2541.0, 2201.0, 1185.0, 1117.0, 1001.0, 708.0, 887.0, 639.0, 671.0, 1039.0, 2657.0, 597.0, 681.0, 135.0, 332.0, 771.0, 1060.0, 984.0, 1061.0, 823.0, 827.0, 318.0, 344.0, 0.0, 245.0, 464.0, 275.0, 1209.0, 1610.0, 284.0, 606.0, 809.0, 830.0, 2732.0],

[2905.0, 2215.0, 2118.0, 2514.0, 2637.0, 2081.0, 1035.0, 961.0, 763.0, 500.0, 667.0, 869.0, 865.0, 782.0, 2613.0, 557.0, 630.0, 353.0, 544.0, 559.0, 896.0, 830.0, 901.0, 770.0, 777.0, 254.0, 278.0, 229.0, 0.0, 310.0, 105.0, 907.0, 1309.0, 283.0, 452.0, 597.0, 596.0, 2696.0],

[2796.0, 2340.0, 2239.0, 2348.0, 2428.0, 2040.0, 1040.0, 936.0, 856.0, 557.0, 721.0, 974.0, 969.0, 839.0, 2526.0, 567.0, 506.0, 470.0, 665.0, 614.0, 870.0, 740.0, 767.0, 482.0, 486.0, 168.0, 163.0, 349.0, 256.0, 0.0, 220.0, 1011.0, 1392.0, 531.0, 473.0, 660.0, 696.0, 2605.0],

[2873.0, 2231.0, 2162.0, 2416.0, 2555.0, 1970.0, 934.0, 859.0, 738.0, 481.0, 646.0, 855.0, 721.0, 767.0, 2584.0, 542.0, 617.0, 337.0, 536.0, 542.0, 806.0, 722.0, 808.0, 653.0, 657.0, 240.0, 263.0, 213.0, 134.0, 209.0, 0.0, 880.0, 1288.0, 359.0, 348.0, 582.0, 581.0, 2668.0],

[3450.0, 1805.0, 1691.0, 2284.0, 2176.0, 1606.0, 519.0, 574.0, 330.0, 627.0, 593.0, 1229.0, 1018.0, 543.0, 3272.0, 1276.0, 1344.0, 1068.0, 1124.0, 485.0, 518.0, 694.0, 775.0, 874.0, 799.0, 976.0, 1002.0, 1082.0, 930.0, 943.0, 902.0, 0.0, 417.0, 946.0, 834.0, 441.0, 730.0, 3328.0],

```
[3838.0, 1615.0, 1482.0, 2347.0, 2277.0, 2080.0, 991.0, 1042.0, 809.0, 1096.0, 1055.0, 1675.0,
1458.0, 1015.0, 3676.0, 1726.0, 1803.0, 1501.0, 1566.0, 961.0, 991.0, 1175.0, 1251.0, 1347.0,
1275.0, 1427.0, 1450.0, 1533.0, 1393.0, 1402.0, 1363.0, 513.0, 0.0, 1395.0, 1301.0, 919.0, 1198.0,
3735.0],
```

```
[3011.0, 2460.0, 2356.0, 2573.0, 2643.0, 2300.0, 1284.0, 1208.0, 1035.0, 830.0, 770.0, 782.0,
852.0, 984.0, 2749.0, 689.0, 774.0, 363.0, 445.0, 869.0, 1150.0, 1081.0, 1154.0, 905.0, 909.0, 396.0,
419.0, 367.0, 303.0, 553.0, 368.0, 1232.0, 1615.0, 0.0, 701.0, 873.0, 871.0, 2827.0],
```

```
[3050.0, 1953.0, 1846.0, 2384.0, 2309.0, 1752.0, 680.0, 733.0, 399.0, 420.0, 555.0, 1140.0, 936.0,
584.0, 2809.0, 997.0, 833.0, 755.0, 887.0, 414.0, 674.0, 702.0, 784.0, 796.0, 725.0, 649.0, 654.0,
631.0, 502.0, 512.0, 474.0, 548.0, 956.0, 703.0, 0.0, 279.0, 245.0, 2889.0],
```

```
[3137.0, 1825.0, 1686.0, 2296.0, 2236.0, 1662.0, 583.0, 632.0, 312.0, 185.0, 264.0, 848.0, 630.0,
387.0, 2886.0, 911.0, 909.0, 672.0, 754.0, 114.0, 582.0, 691.0, 773.0, 851.0, 774.0, 600.0, 626.0,
695.0, 491.0, 501.0, 459.0, 448.0, 855.0, 574.0, 466.0, 0.0, 349.0, 2967.0],
```

```
[2930.0, 2020.0, 1896.0, 2365.0, 2294.0, 1737.0, 652.0, 665.0, 462.0, 528.0, 611.0, 1193.0, 987.0,
631.0, 2650.0, 821.0, 652.0, 631.0, 834.0, 468.0, 632.0, 531.0, 613.0, 583.0, 559.0, 530.0, 551.0,
507.0, 414.0, 439.0, 304.0, 592.0, 999.0, 652.0, 200.0, 357.0, 0.0, 2728.0],
```

```
[698.0, 3115.0, 3331.0, 3673.0, 3712.0, 3830.0, 3453.0, 3371.0, 3327.0, 3193.0, 3289.0, 3408.0,
3240.0, 3311.0, 544.0, 2737.0, 2500.0, 2828.0, 2991.0, 3203.0, 3305.0, 3303.0, 2978.0, 2847.0,
2858.0, 2776.0, 2797.0, 2705.0, 2878.0, 2785.0, 2851.0, 3450.0, 3829.0, 2955.0, 2970.0, 3247.0,
3175.0, 0.0]
```

```
]
```

```
# Cast matrix to Integers
```

```
data["time_matrix"] = [[int(x) for x in row] for row in raw_matrix]
```

```
#Hardcoded time windows
```

```
data["time_windows"] = [
```

```
(0, 52800), # 0. 4:20 to 19:00
```

```
(0, 15000), # 1. 8:30
```

```
(0, 13200), # 2. 8:00
```

```
(0, 16800), # 3. 9:00
```

```
(0, 6000), # 4. 6:00
```

```
(0, 7800), # 5. 6:30
```

```
(0, 13200), # 6. 8:00
```

```
(0, 52800), # 7. TK 19:00 (7:00 PM)
```

(0, 13200), # 8. 8:00
 (0, 13200), # 9. 8:00
 (0, 52800), # 10. TK 19:00 (7:00 PM)
 (0, 16800), # 11. 9:00
 (0, 13200), # 12. 8:00
 (0, 13200), # 13. 8:00
 (0, 3300), # 14. 5:15
 (0, 16800), # 15. 9:00
 (0, 16800), # 16. 9:00
 (0, 13200), # 17. 8:00
 (0, 6000), # 18. Originally 5:00; Extended to 6:00 AM
 (0, 9600), # 19. 7:00
 (0, 7800), # 20. 6:30
 (0, 25800), # 21. 11:30
 (0, 11400), # 22. 7:30
 (0, 52800), # 23. 19:00 (7:00 PM)
 (0, 9000), # 24. 6:50
 (0, 9600), # 25. 7:00
 (0, 52800), # 26. TK 19:00 (7:00 PM)
 (0, 13200), # 27. 8:00
 (0, 13200), # 28. 8:00
 (11400, 13200), # 29. "7:30 to 8:00 AM" (Strict Window)
 (13200, 16800), # 30. "8:00 to 9:00 AM" (Not before 8:00 and 9 is opening time)
 (0, 52800), # 31. TK 19:00 (7:00 PM)
 (0, 52800), # 32. TK 19:00 (7:00 PM)
 (0, 52800), # 33. TK 19:00 (7:00 PM)
 (0, 52800), # 34. TK 19:00 (7:00 PM)
 (0, 52800), # 35. TK 19:00 (7:00 PM)
 (0, 52800), # 36. TK 19:00 (7:00 PM)
 (0, 16800) # 37. 9:00

```

]

data["num_vehicles"] = 2

data["depot"] = 0

return data


#Displays solution with Waiting Time (Slack)
def print_solution_with_slack(data, manager, routing, solution):

    base_time = datetime(2025, 1, 1, 4, 20, 0)

    schedule_data = {}

    time_dimension = routing.GetDimensionOrDie("Time")


    print(f"Objective Value: {solution.ObjectiveValue()}")


    for vehicle_id in range(data["num_vehicles"]):

        index = routing.Start(vehicle_id)

        prev_index = index # Keep track of previous node


        col_nodes = []
        col_times = []
        col_waits = []


        while not routing.IsEnd(index):

            node_index = manager.IndexToNode(index)

            time_var = time_dimension.CumulVar(index)

            current_seconds = solution.Min(time_var)


            # Calculating wait time

            wait_time_min = 0

```



```

if index != routing.Start(vehicle_id):

    # previous time

    prev_time_var = time_dimension.CumulVar(prev_index)

    prev_seconds = solution.Min(prev_time_var)


    # travel time

    prev_node = manager.IndexToNode(prev_index)

    curr_node = manager.IndexToNode(index)

    travel_seconds = data["time_matrix"][prev_node][curr_node]


    # Calculating Arrival

    arrival_seconds = prev_seconds + travel_seconds


    # Wait = Start - Arrival

    wait_seconds = current_seconds - arrival_seconds

    wait_time_min = int(wait_seconds / 60) # Convert to minutes


    clock_time = base_time + timedelta(seconds=int(current_seconds))


    col_nodes.append(node_index)

    col_times.append(clock_time.strftime("%H:%M"))

    # Show wait only if it exists

    col_waits.append(f"{wait_time_min} min" if wait_time_min > 0 else "-")


    prev_index = index

    index = solution.Value(routing.NextVar(index))


# Depot Return

```

```

node_index = manager.IndexToNode(index)
time_var = time_dimension.CumulVar(index)
current_seconds = solution.Min(time_var)

# Calculating wait at end
prev_time_var = time_dimension.CumulVar(prev_index)
prev_seconds = solution.Min(prev_time_var)
prev_node = manager.IndexToNode(prev_index)
curr_node = manager.IndexToNode(index)
travel_seconds = data["time_matrix"][prev_node][curr_node]
arrival_seconds = prev_seconds + travel_seconds
wait_seconds = current_seconds - arrival_seconds
wait_time_min = int(wait_seconds / 60)

clock_time = base_time + timedelta(seconds=int(current_seconds))

col_nodes.append(node_index)
col_times.append(clock_time.strftime("%H:%M"))
col_waits.append(f"{wait_time_min} min" if wait_time_min > 0 else "-")

# Store Columns
schedule_data[f"V{vehicle_id} Node"] = pd.Series(col_nodes)
schedule_data[f"V{vehicle_id} Time"] = pd.Series(col_times)
schedule_data[f"V{vehicle_id} Wait"] = pd.Series(col_waits)

# Display
df = pd.DataFrame(schedule_data).fillna("")
display(df)

# CSV Block

```

```

print("\n" + "="*40)
print(" COPY FOR SPREADSHEET (CSV)")
print("="*40)
print(df.to_csv(index=False, sep=','))

# 2. CREATE DATAFRAME
df = pd.DataFrame(schedule_data)

# Fill empty cells (if one route is shorter than the other)
df = df.fillna("")

# Display nicely in Colab
display(df)

def main():

    data = create_data_model()

    # Create the routing index manager
    manager = pywrapcp.RoutingIndexManager(
        len(data["time_matrix"]), data["num_vehicles"], data["depot"]
    )

    # Create Routing Model
    routing = pywrapcp.RoutingModel(manager)

    # Return the travel time between the two nodes
    def time_callback(from_index, to_index):
        # Convert from routing variable Index to time matrix NodeIndex.

```

```

from_node = manager.IndexToNode(from_index)
to_node = manager.IndexToNode(to_index)
return data["time_matrix"][from_node][to_node]

transit_callback_index = routing.RegisterTransitCallback(time_callback)

# Define cost of each arc.
routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index)

# Configure Time Dimension
time = "Time"
routing.AddDimension(
    transit_callback_index,
    72000, # allow waiting time
    72000, # maximum time per vehicle
    False, # Don't force start cumul to zero.
    time,
)
time_dimension = routing.GetDimensionOrDie(time)

# Add time window constraints for each location except depot
for location_idx, time_window in enumerate(data["time_windows"]):
    if location_idx == data["depot"]:
        continue
    index = manager.NodeToIndex(location_idx)
    time_dimension.CumulVar(index).SetRange(time_window[0], time_window[1])

# Add time window constraints for each vehicle start node
depot_idx = data["depot"]
for vehicle_id in range(data["num_vehicles"]):

```

```

index = routing.Start(vehicle_id)
time_dimension.CumulVar(index).SetRange(
    data["time_windows"][depot_idx][0], data["time_windows"][depot_idx][1]
)

# Instantiate route start and end times to produce feasible times
for i in range(data["num_vehicles"]):
    routing.AddVariableMinimizedByFinalizer(
        time_dimension.CumulVar(routing.Start(i))
    )
    routing.AddVariableMinimizedByFinalizer(time_dimension.CumulVar(routing.End(i)))

# penalty for dropping locations if the solution is impossible
penalty = 1000000
for node in range(1, len(data["time_matrix"])):
    routing.AddDisjunction([manager.NodeToIndex(node)], penalty)

# Setting first solution heuristic.
search_parameters = pywrapcp.DefaultRoutingSearchParameters()
search_parameters.time_limit.seconds = 60
search_parameters.log_search = True
#First Search Strategy
search_parameters.first_solution_strategy = (
    routing_enums_pb2.FirstSolutionStrategy.GLOBAL_CHEAPEST_ARC
)
#Local Search Metaheuristic Strategy
search_parameters.local_search_metaheuristic = (
    routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
)

```

```
# Solve the problem.
solution = routing.SolveWithParameters(search_parameters)

# Print solution on console.
if solution:
    print_solution_with_slack(data, manager, routing, solution)
else:
    print ("No Solution found")

main()
```